

UNIDAD 3: ANÁLISIS EXPLORATORIO DE DATOS

Contenido

CAPÍTULO 1.- IMPORTANCIA DE EDA	3
1.1.- ¿Por qué es crucial el EDA?	3
1.2.- Ejemplo de Código: EDA simple usando Pandas	4
1.3.- Importancia en Big Data.....	6
1.4.- Human Element (elemento humano)	6
1.5.- Risk Mitigation.....	6
1.6.- Ejemplos de Diferentes Sectores	7
1.7.- Comparación de conjuntos de datos	7
1.8.- Fragmentos de código para EDA Visual	8
CAPÍTULO 2.- TIPOS DE DATOS	8
2.1.- Datos Numéricos	8
2.2.- Datos Categóricos.....	10
2.3.- Datos Textuales	11
2.4.- Datos de Series Temporales	12
2.5 Datos Multivariados.....	13
2.6.- Datos Geoespaciales	13
CAPÍTULO 3.- ESTADÍSTICA DESCRIPTIVA.....	15
3.1.- ¿Qué son las estadísticas descriptivas?	15
3.2.- Medidas de Tendencia Central	15
3.3.- Medidas de Variabilidad.....	16
3.4.- ¿Por qué es útil?	17
3.5.- Ejemplo: Examinando datos de ventas.....	17
3.6.- Ejemplo: Examinando reseñas de clientes	18
3.7.- Asimetría y Curtosis	19
3.8.- Conclusiones sobre la importancia del EDA	20
CAPÍTULO 4.- PREPROCESAMIENTO DE DATOS	20
4.1.- Limpieza de datos	20



CAPÍTULO 5: ANÁLISIS EXPLORATORIO DE DATOS VISUAL	21
5.1 ANÁLISIS UNIVARIADO	21
5.1.1 Histogramas	22
5.1.2 Diagramas de Caja	23
5.1.3 Gráficos de conteo para datos categóricos	25
5.1.4 Estadísticas descriptivas junto con Visualizaciones.....	27
5.1.5 Gráfico de Densidad de Kernel	29
5.1.6 Diagrama de Violín	32
5.1.7 Sesgo y Curtosis de los Datos	33
5.2 ANÁLISIS BIVARIADO	35
5.2.1 Gráficos de Dispersión	36
5.2.2 Coeficiente de Correlación	37
5.2.3 Gráficos de Líneas.....	37
5.2.4 Mapas de Calor.....	39
5.2.5 Pairplots	41
5.2.6 Significancia Estadística en el Análisis Bivariado	42
5.2.7 Manejo de Variables Categóricas en el Análisis Bivariado.....	42
5.2.8 Aplicaciones del Análisis Bivariado en el Mundo Real	43
5.3 ANÁLISIS MULTIVARIADO	44
5.3.1 ¿Qué es el Análisis Multivariado?.....	44
5.3.2 Tipos de Análisis Multivariado	45
5.3.4 Ejemplo: Análisis de Clústeres	45
5.3.5 Aplicaciones del Análisis Multivariado en el Mundo Real.....	47
5.3.6 Mapas de Calor para Matrices de Correlación	48
5.3.7 Ejemplo utilizando Análisis de Regresión Múltiple	49
5.3.8 Puntos de Precaución	51
5.3.9 Otras Técnicas de Reducción de Dimensionalidad	51
EJEMPLOS PRÁCTICOS CAPÍTULO 5	52



CAPÍTULO 1.- IMPORTANCIA DE EDA

Si piensas en el análisis de datos como una búsqueda del tesoro, entonces el Análisis Exploratorio de Datos (EDA) es el mapa del tesoro que guía tu camino. El EDA proporciona una visión completa de tus datos, incluyendo sus dimensiones, características y patrones ocultos. Al tener este “mapa” de tus datos antes de decidir la mejor “ruta” para encontrar ideas, puedes comprender mejor lo que te espera.

Supongamos que tienes un conjunto de datos que registra el comportamiento del cliente en una tienda minorista. Simplemente mirar los datos sin procesar no te dará ideas reales. Sin embargo, con el EDA, puedes responder preguntas como: ¿Hay un patrón en cuándo aumentan las ventas? ¿Cuál es la edad promedio de los clientes? ¿Las personas que compran el Producto A también tienden a comprar el Producto B? Al encontrar las respuestas a estas preguntas, puedes obtener una mejor comprensión de tus datos y, en última instancia, tomar decisiones más informadas.

1.1. - ¿Por qué es crucial el EDA?

Limpieza de Datos

El Análisis Exploratorio de Datos (EDA) es un paso crucial en cualquier proyecto de ciencia de datos, ya que te permite obtener una comprensión más profunda de tus datos e identificar patrones o relaciones que pueden no ser obvios de inmediato.

Además de ayudarte a identificar valores atípicos, valores faltantes o errores humanos que pueden necesitar atención antes del análisis, el EDA también te permite comprender la distribución de tus datos, evaluar la calidad de tus variables y determinar cualquier problema potencial con tu proceso de recolección de datos. Al realizar un EDA completo, puedes asegurarte de que tus datos estén limpios, sean confiables y estén listos para el análisis, lo que en última instancia conducirá a ideas más precisas y accionables.

Pruebas de Supuestos

Los modelos estadísticos se construyen sobre ciertos supuestos sobre los datos. Estos supuestos a menudo se hacen sobre la distribución y la variabilidad de los datos. Sin embargo, no siempre está claro si los datos cumplen con estos supuestos.

Aquí es donde entra en juego el análisis exploratorio de datos (EDA). El EDA es un proceso de examinar los datos para comprender mejor sus propiedades y detectar cualquier patrón o anomalía que pueda estar presente. Al realizar un EDA, podemos verificar si los supuestos hechos sobre los datos son válidos o no. Esto ayuda a garantizar que los modelos estadísticos que construimos sean precisos y confiables.

Ingeniería de Características



El Análisis Exploratorio de Datos (EDA) es un paso esencial en el proceso de desarrollo de un modelo de aprendizaje automático. Durante esta etapa, es posible que descubras que ciertas características requieren transformación, escalamiento o incluso creación para mejorar la precisión del modelo.

Por ejemplo, puedes encontrar que una característica en particular tiene valores atípicos que necesitan ser identificados y tratados, o que ciertas características están altamente correlacionadas y necesitan ser combinadas en una sola característica. Además, el EDA puede ayudarte a identificar patrones en los datos que pueden informar la selección de modelos y algoritmos apropiados, o llevar al descubrimiento de nuevas variables que pueden ser relevantes para el problema en cuestión.

Selección de Modelo

El Análisis Exploratorio de Datos (EDA) es una fase crítica en la preparación de datos para modelos de aprendizaje automático. Este proceso implica identificar patrones, tendencias y relaciones en los datos que pueden proporcionar ideas valiosas sobre los factores que influyen en la variable de resultado.

Al explorar los datos de esta manera, puedes obtener una comprensión más profunda de la naturaleza subyacente de los datos y identificar cualquier problema potencial que pueda necesitar ser abordado antes del modelado.

Además, los resultados del EDA pueden ayudarte a seleccionar el modelo de aprendizaje automático más apropiado para tu problema particular. Por lo tanto, dedicar tiempo a realizar EDA es un paso esencial en cualquier proyecto de ciencia de datos que involucre aprendizaje automático.

Información para Negocios

El Análisis Exploratorio de Datos (EDA) es una herramienta crucial que puede ayudar a las empresas a obtener ideas valiosas. Al analizar datos, el EDA puede revelar información importante sobre el negocio minorista, como los mejores meses para las ventas, los patrones de compra de los clientes o incluso las ineficiencias en la cadena de suministro. Con esta información, las empresas pueden tomar decisiones basadas en datos para mejorar sus operaciones, aumentar la eficiencia y maximizar las ganancias.

Además, el EDA puede proporcionar una comprensión más profunda del comportamiento, las preferencias y las necesidades de los clientes, lo que puede llevar al desarrollo de mejores productos y servicios que satisfagan sus necesidades. En resumen, el EDA desempeña un papel esencial en ayudar a las empresas a comprender sus datos, obtener ideas valiosas y tomar decisiones informadas para optimizar su rendimiento y éxito.

1.2.- Ejemplo de Código: EDA simple usando Pandas

Para comenzar nuestra exploración de datos, utilizaremos Pandas, una biblioteca de Python versátil y poderosa. Para realizar un análisis preliminar en un conjunto de datos hipotético de ventas minoristas.



El conjunto de datos puede contener información como el nombre del producto, la cantidad vendida y la fecha de compra.

Usando Pandas, podemos cargar y visualizar fácilmente los datos, obtener información sobre las tendencias de ventas e identificar áreas para un análisis más detallado. Por ejemplo, podemos identificar los productos más vendidos o explorar la relación entre el precio y la cantidad vendida. En general, Pandas nos proporciona un conjunto de herramientas valiosas para analizar nuestros datos de ventas minoristas y tomar decisiones comerciales informadas.

Ejemplo

```
import pandas as pd
```

```
# Load the dataset
```

```
df = pd.read_csv('retail_sales.csv')
```

```
# Get a sense of the data
```

```
print(df.head())
```

```
# Summary statistics
```

```
print(df.describe())
```

```
# Checking for missing values
```

```
print(df.isnull().sum())
```

```
# Frequency of sales in each month
```

```
print(df['Month'].value_counts())
```

El Análisis Exploratorio de Datos (EDA) es un componente vital de cualquier proyecto de ciencia de datos. Implica un enfoque sistemático para analizar y comprender los datos, lo cual es esencial para obtener ideas significativas y tomar decisiones informadas. Aunque la afirmación anterior pueda parecer que cubre todo, en realidad el EDA es un proceso complejo que requiere el uso de múltiples herramientas y técnicas. En este capítulo exploraremos algunas de estas herramientas y técnicas con más detalle para darte una mejor comprensión de cómo realizar un EDA exitoso.

Es importante señalar que el EDA no es un proceso que se realiza una sola vez. Más bien, es un proceso iterativo y creativo que requiere un diálogo continuo con tus datos. Cada vez que encuentres nuevos datos, necesitarás revisar tu proceso de EDA para asegurarte de que estás descubriendo nuevas ideas y tomando decisiones informadas. Esto significa que el EDA no es solo un paso en el proceso, sino una conversación continua con tus datos y un componente crítico de cualquier proyecto exitoso de ciencia de datos.



1.3.- Importancia en Big Data

En el mundo actual, donde los datos están en todas partes, la importancia del “Big Data” no puede ser subestimada. Tener una gran cantidad de datos puede ser beneficioso para obtener conclusiones más precisas, pero también puede traer desafíos, como lidiar con el “big noise”. El ruido es la información irrelevante o redundante en los datos que puede distorsionar el análisis.

1.4.- Human Element (elemento humano)

El aprendizaje automático (Machine learning) y la inteligencia artificial han revolucionado el análisis de datos, pero es importante señalar que el “toque humano” sigue desempeñando un papel crucial. Aunque la IA puede procesar grandes cantidades de datos de manera rápida y precisa, carece de la intuición que proviene de años de experiencia y conocimiento.

Durante el análisis exploratorio de datos (EDA), es esencial que los analistas humanos aporten su perspectiva única. Por ejemplo, mientras que una máquina puede tener dificultades para diferenciar entre asociación y correlación en un conjunto de variables, un analista humano puede percibir intuitivamente la relación y ofrecer interpretaciones más matizadas.

En resumen, aunque la tecnología ha impulsado enormemente el campo del análisis de datos, es la experiencia humana la que realmente puede desbloquear todo su potencial.

1.5.- Risk Mitigation

El Análisis Exploratorio de Datos (EDA) puede ser una herramienta de mitigación de riesgos altamente efectiva, especialmente en sectores cruciales como las finanzas y la salud. Al aprovechar el EDA, las industrias pueden identificar posibles problemas o valores atípicos que, de otro modo, podrían pasar desapercibidos. Este proceso puede ayudar a detectar actividades fraudulentas en transacciones financieras, las cuales pueden prevenirse o mitigarse.

Además, en el ámbito sanitario, el EDA puede utilizarse para detectar datos anómalos de pacientes, lo que podría conducir al diagnóstico temprano de afecciones graves. Esto puede facilitar la prestación de asistencia médica oportuna y mejorar los resultados para los pacientes.

Asimismo, el EDA también puede revelar patrones y tendencias que no son evidentes de inmediato, permitiendo que las organizaciones tomen decisiones basadas en datos que mejoren su rentabilidad o su eficacia general.

Example

```
# Simple code to identify outliers in a dataset
import numpy as np

data = np.array([1, 2, 3, 50, 5, 6, 7])
mean = np.mean(data)
std_dev = np.std(data)
```



```
# Identifying outliers
outliers = [x for x in data if abs(x - mean) >= 2 * std_dev]

print("Outliers:", outliers)
```

1.6.- Ejemplos de Diferentes Sectores

La versatilidad del EDA es verdaderamente notable y esto se puede ver en su amplio uso en una variedad de sectores. Por ejemplo, en la industria del comercio electrónico, el EDA desempeña un papel crítico en el seguimiento del comportamiento del usuario, permitiendo a las empresas identificar tendencias y patrones clave que pueden informar las estrategias de marketing y ventas.

De manera similar, en el sector de la salud, el EDA es una herramienta vital para analizar datos importantes de pacientes, como signos vitales, lo que permite a los profesionales médicos tomar decisiones mejor informadas sobre la atención al paciente. Con su capacidad para descubrir ideas y tendencias valiosas en los datos, el EDA se ha convertido en un primer paso esencial para tomar decisiones basadas en datos en muchos sectores e industrias.

1.7.- Comparación de conjuntos de datos

Cuando se trata de análisis de datos, no es raro tener datos de diferentes períodos o departamentos que necesiten ser comparados. El Análisis Exploratorio de Datos (EDA) puede ayudarte a obtener información sobre la compatibilidad de estos conjuntos de datos. Con el EDA, puedes determinar si los conjuntos de datos deben ser analizados por separado o si pueden fusionarse para un análisis más completo.

Además, el EDA también puede brindarte una comprensión más profunda de los conjuntos de datos individuales y ayudarte a identificar cualquier patrón o tendencia subyacente que pueda no ser evidente de inmediato. Al realizar un EDA completo, puedes asegurarte de estar tomando las decisiones más informadas basadas en los datos disponibles, lo que conduce a mejores resultados.

Ejemplo:

```
# Código en Python para comparar dos conjuntos de datos usando medidas estadísticas simples
data1 = np.array([1, 2, 3, 4, 5])
data2 = np.array([2, 3, 4, 5, 6])

mean1, mean2 = np.mean(data1), np.mean(data2)
std_dev1, std_dev2 = np.std(data1), np.std(data2)

print("Media del conjunto de datos 1:", mean1)
print("Media del conjunto de datos 2:", mean2)
print("Standard Deviation of Dataset 1:", std_dev1)
print("Standard Deviation of Dataset 1:", std_dev2)
```



1.8.- Fragmentos de código para EDA Visual

El EDA visual es una herramienta indispensable cuando se trata de analizar datos. De hecho, a menudo se dice que una imagen vale más que mil palabras. Al usar gráficos simples como histogramas, diagramas de caja o gráficos de dispersión, podemos obtener ideas instantáneas sobre los datos e identificar patrones que podrían no ser evidentes al observar los datos en bruto.

Además, el EDA visual puede ayudarnos a detectar valores atípicos, explorar relaciones entre variables e incluso identificar áreas potenciales para análisis adicional. En resumen, no se puede negar que el EDA visual es una técnica poderosa que puede ayudarnos a comprender mejor nuestros datos y tomar decisiones más informadas basadas en nuestros hallazgos.

Ejemplo

```
# Simple code for histogram using matplotlib
import matplotlib.pyplot as plt

data = [2, 3, 4, 4, 5, 6, 6, 7, 8, 9]
plt.hist(data, bins=5, alpha=0.5, color='blue')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Simple Histogram')
plt.show()
```

Ahora, discutiremos los diferentes tipos de datos que encontrarás comúnmente. Comprender la naturaleza de tus datos es crucial para un EDA efectivo, ya que te guiará en la selección de las herramientas y técnicas apropiadas para la exploración y el análisis. Vamos a categorizar los tipos de datos en grupos principales y proporcionar ejemplos para facilitar su comprensión.

CAPÍTULO 2.- TIPOS DE DATOS

2.1.- Datos Numéricos

Los datos numéricos son un elemento esencial de la investigación científica y representan mediciones cuantitativas de diversos fenómenos. Se dividen en dos tipos principales: datos discretos y continuos. Los datos discretos se refieren a datos que solo pueden tomar ciertos valores específicos y a menudo se obtienen mediante el conteo.

Por ejemplo, el número de autos en un estacionamiento se puede contar, y el resultado es un número discreto. Por otro lado, los datos continuos se refieren a datos que pueden tomar cualquier valor dentro de un rango específico y pueden medirse utilizando una escala. Por ejemplo, el peso de un objeto se puede medir utilizando una balanza, y el resultado es un dato continuo. Ambos tipos de datos son importantes en la investigación científica y pueden proporcionar información valiosa sobre diversos fenómenos.



Datos Discretos

Este tipo de datos consiste en valores distintos y separados que no pueden subdividirse en unidades más pequeñas. A menudo está compuesto por recuentos de cosas que son fácilmente medibles. Un buen ejemplo de datos discretos es el número de empleados en una empresa.

Sin embargo, es importante tener en cuenta que los datos discretos también pueden incluir otros tipos de información como grupos de edad, tallas de zapatos y el número de estudiantes en un aula. El análisis de datos discretos implica determinar la frecuencia de ocurrencia de cada valor e identificar patrones y tendencias que surgen.

Este tipo de datos es extremadamente útil en varios campos como estadística, finanzas y marketing, donde se utiliza para obtener ideas significativas y tomar decisiones informadas.

Datos Continuos

Estos son puntos de datos que pueden tomar cualquier valor dentro de un rango. Los datos continuos pueden expresarse en valores decimales o fraccionarios. Los datos continuos pueden medirse con un alto grado de precisión, por lo que se utilizan con frecuencia en la investigación científica. La altura, el peso y la temperatura son ejemplos de datos continuos.

Además, otros ejemplos de datos continuos incluyen la distancia, el tiempo y la edad. Los datos continuos pueden subdividirse aún más en dos tipos: datos de intervalo y datos de razón. Los datos de intervalo se refieren a datos que no tienen un punto cero verdadero, mientras que los datos de razón se refieren a datos que sí tienen un punto cero verdadero.

Ejemplo

Código de ejemplo para graficar datos discretos y continuos

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# Datos discretos
```

```
discrete_data = np.random.choice([1, 2, 3, 4, 5], 50)
```

```
plt.subplot(1, 2, 1)
```

```
plt.hist(discrete_data, bins=5)
```

```
plt.title('Datos Discretos')
```

```
import numpy as np
```

```
data = np.random.normal(2, 2, 50)
```

```
bins = 5
```

```
histogram = np.histogram(data, bins=5)
```

```
print(histogram)
```

2.2.- Datos Categóricos

Los datos categóricos son un tipo de datos que se utiliza para representar diferentes características o etiquetas. Los datos categóricos pueden dividirse en dos categorías, a saber, datos nominales y ordinales. Las categorías nominales se utilizan para representar datos que no tienen un orden inherente, como los colores de un arcoíris o las diferentes razas de perros.

Por otro lado, las categorías ordinales se utilizan para representar datos que tienen un orden inherente, como los diferentes tamaños de camisetas (pequeño, mediano, grande). Es importante tener en cuenta que los datos categóricos pueden ser útiles en muchos campos diferentes, como el marketing, las ciencias sociales y el análisis de datos.

Datos Nominales

Los datos nominales no tienen un orden o clasificación natural. Ejemplos incluyen colores, género y tipos de frutas. Los datos nominales son un tipo de datos que no tienen un orden o clasificación natural. Cada valor se considera una categoría distinta y separada en los datos, y cada valor se considera igual. Por ejemplo, cuando recopilamos datos sobre colores, género o tipos de frutas, estamos recopilando datos nominales.

Una característica común en los datos nominales es considerar las categorías que representan los datos como una categoría distinta y separada de las demás, lo que significa que no hay una jerarquía o un orden entre las categorías. Por ejemplo, cuando recopilamos datos sobre los colores de una camiseta, no clasificamos un color como mejor o peor que otro. Más bien, cada color se considera una categoría separada.

Es importante tener en cuenta que los datos nominales no son el único tipo de datos que existen. También existen otros tipos de datos, incluyendo datos ordinales, de intervalo y de razón. Cada uno de estos tipos de datos tiene sus propias propiedades y características únicas, que los hacen útiles en diferentes contextos de análisis.

En resumen, los datos nominales son un tipo de datos que no tienen un orden o clasificación natural. Ejemplos de datos nominales incluyen colores, género y tipos de frutas.

Datos Ordinales

Este tipo de datos tiene un orden natural en el que se organizan las categorías, pero los intervalos entre las categorías no son iguales. Se utiliza para representar datos que implican juicios subjetivos, como las calificaciones de satisfacción del cliente.

En este caso, los datos se pueden clasificar en categorías como *“Pobre”*, *“Promedio”* y *“Excelente”*. Los datos ordinales también se pueden utilizar para representar datos de encuestas donde los encuestados califican su nivel de acuerdo con una afirmación utilizando una escala como *“Totalmente en desacuerdo”*, *“En desacuerdo”*, *“Neutral”*, *“De acuerdo”* y *“Totalmente de acuerdo”*. Dado que las



categorías están clasificadas, pero los intervalos entre ellas no son uniformes, los datos ordinales pueden ser complicados de analizar.

Por lo tanto, es importante elegir un método estadístico adecuado para analizar este tipo de datos.

Ejemplo

```
# Example code to plot nominal and ordinal data using bar plots
import seaborn as sns

# Nominal Data
sns.countplot(x=["Apple", "Banana", "Apple", "Orange", "Banana", "Apple", "Orange"])
plt.title('Nominal Data')
plt.show()

# Ordinal Data
sns.countplot(x=["Poor", "Average", "Excellent", "Poor", "Average"])
plt.title('Ordinal Data')
plt.show()
```

2.3.- Datos Textuales

Los datos textuales se refieren a cualquier tipo de datos no estructurados, como publicaciones en redes sociales, comentarios y artículos de noticias. Estos tipos de datos tradicionalmente no se analizaban con EDA, pero con los avances en Procesamiento del Lenguaje Natural (NLP, por sus siglas en inglés), ahora es posible extraer información significativa de los datos textuales.

Las técnicas de NLP pueden utilizarse para identificar patrones y tendencias en grandes cantidades de datos de texto. Además, se puede realizar análisis de sentimientos para comprender el tono emocional del texto y categorizarlo en positivo, negativo o neutral.

Esto permite a empresas y organizaciones comprender mejor la retroalimentación de los clientes y el sentimiento general. Los datos textuales pueden utilizarse para detectar temas y problemas emergentes, lo que puede ayudar a las empresas a mantenerse a la vanguardia y responder de manera proactiva a las tendencias cambiantes.

Ejemplo

```
# Simple example using word frequency
from collections import Counter

text_data = "Exploratory Data Analysis is important for data science."
word_count = Counter(text_data.split())
print("Word Frequency:", word_count)
```



2.4.- Datos de Series Temporales

Los datos de series temporales se refieren a un tipo particular de datos que se recopilan o registran en puntos sucesivos en el tiempo. Estos puntos de datos pueden capturarse en intervalos regulares o irregulares y a menudo se utilizan para analizar patrones o tendencias a lo largo del tiempo.

Una aplicación práctica de los datos de series temporales se encuentra en el mercado de valores, donde se rastrean los precios de las acciones y otros instrumentos financieros a lo largo del tiempo para informar las decisiones de inversión. Otro ejemplo es el de los datos meteorológicos, que se recopilan en intervalos regulares para monitorear los cambios en la temperatura, la precipitación y otros fenómenos meteorológicos.

En los últimos años, el crecimiento explosivo de las redes sociales también ha dado lugar a la creación de vastas cantidades de datos de series temporales. Por ejemplo, los datos de actividad de Twitter pueden analizarse para rastrear cambios en la opinión pública o para identificar tendencias y temas emergentes.

En general, el uso de datos de series temporales en una variedad de campos se ha vuelto cada vez más importante, ya que proporciona una herramienta valiosa para comprender y predecir patrones a lo largo del tiempo.

Ejemplo

```
# Simple time-series plot
import pandas as pd

time_series_data = pd.DataFrame({
    'Date': pd.date_range(start='1/1/2022', periods=10, freq='D'),
    'Stock_Price': [1, 2, 3, 4, 3, 4, 5, 6, 7, 8]
})

time_series_data.plot(x='Date', y='Stock_Price', kind='line')
plt.title("Time Series Data")
plt.show()
```

Entender los diversos tipos de datos es un aspecto fundamental del análisis exploratorio de datos (EDA, por sus siglas en inglés). Implica aprender a visualizar y gestionar datos de manera efectiva, lo cual es crucial para tu trayectoria de exploración de datos. En las secciones siguientes, proporcionaremos ideas detalladas sobre cómo cada tipo de datos requiere un enfoque distinto para un análisis efectivo.

Al dominar estas técnicas, estarás bien preparado para manejar conjuntos de datos intrincados y sacar conclusiones significativas de ellos. Esto te permitirá obtener ideas valiosas y tomar decisiones informadas en diversos campos, incluyendo negocios, finanzas, atención médica y más



2.5 Datos Multivariados

En el análisis exploratorio de datos (EDA), los **datos multivariados** se refieren a conjuntos de datos que contienen **más de dos variables**. Analizar varias variables simultáneamente permite descubrir **patrones, tendencias o correlaciones** que no serían evidentes si se estudiaran de forma aislada.

Una técnica común para visualizar relaciones entre múltiples variables es el **pairplot**, que genera una **matriz de gráficos de dispersión** para cada par de variables. Esto facilita:

- Identificar relaciones lineales o no lineales.
- Detectar correlaciones.
- Observar posibles valores atípicos.

Además de los pair plots, las técnicas de análisis de datos multivariados se pueden utilizar para desarrollar modelos que pueden predecir resultados basados en la relación entre múltiples variables. Estos modelos pueden utilizarse para prever tendencias, identificar patrones y tomar decisiones informadas. Al utilizar el análisis de datos multivariados, uno puede obtener una comprensión más completa de conjuntos de datos complejos y tomar decisiones informadas basadas en las relaciones entre múltiples variables.

A continuación, se muestra un ejemplo en Python utilizando la librería **Seaborn** para crear un pairplot:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
# Create a DataFrame
df = pd.DataFrame({
    'Height': [5.9, 5.8, 5.6, 6.1, 5.7],
    'Weight': [75, 80, 77, 89, 94],
    'Age': [21, 22, 20, 19, 18]
})
# Create a pairplot
sns.pairplot(df)
plt.suptitle('Multivariate Data Visualization', y=1.02)
plt.show()
```

En el pairplot anterior, puedes examinar visualmente cómo **Altura, Peso y Edad** interactúan entre sí. Esto puede ser muy útil para identificar patrones o anomalías en los datos.

2.6.- Datos Geoespaciales

Los datos geoespaciales son un tipo de datos que contienen información sobre la ubicación geográfica de objetos o eventos. Este tipo de datos es altamente valioso ya que proporciona una amplia gama de información que puede ser utilizada en diversos campos.



Por ejemplo, puede proporcionar información detallada sobre los patrones climáticos de una región particular, la ubicación de los recursos naturales y la densidad de población de un área. Estos datos también pueden ser utilizados para estudiar el impacto de las actividades humanas en el medio ambiente y desarrollar estrategias para mitigarlos.

La complejidad de los datos geoespaciales puede variar ampliamente, desde simples coordenadas de latitud y longitud de una ciudad hasta un mapa de varias capas que contiene una amplia gama de información. En general, los datos geoespaciales son una herramienta esencial en muchas industrias y desempeñan un papel crucial en nuestra comprensión del mundo que nos rodea.

Aquí tienes un ejemplo simple que representa las coordenadas geográficas (latitud y longitud) de tres ciudades: Nueva York, Los Ángeles y Chicago.

```
import matplotlib.pyplot as plt

# Sample coordinates: [latitude, longitude]
locations = [
    [40.7128, -74.0060], # New York
    [34.0522, -118.2437], # Los Angeles
    [41.8781, -87.6298], # Chicago
]

# Unzip the coordinates
latitudes, longitudes = zip(*locations)
# Create a scatter plot
plt.scatter(longitudes, latitudes)
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Geospatial Data Visualization")
plt.show()
```

Este es un ejemplo básico que se puede ampliar de varias maneras para mejorar su funcionalidad y utilidad. Por ejemplo, puedes incluir capas adicionales como carreteras, puntos de referencia u otros datos relevantes que podrían ser útiles para tu aplicación específica.

Al introducir estos tipos adicionales de datos, puedes obtener una comprensión más completa de los tipos de datos que podrías encontrar en escenarios reales de análisis geoespacial. Esto puede ayudarte a prepararte mejor para tales escenarios y desarrollar modelos de análisis de datos más precisos y fiables. Además, al incorporar más capas de datos en tu análisis, también puedes aumentar la profundidad y complejidad de tu análisis, lo que te permitirá descubrir ideas y tendencias que podrían no ser evidentes en un análisis más básico.



CAPÍTULO 3.- ESTADÍSTICA DESCRIPTIVA

Bienvenido a esta sección dedicada al apasionante campo de las **Estadísticas Descriptivas**, un pilar fundamental dentro del Análisis Exploratorio de Datos (EDA). Si alguna vez has cursado una asignatura introductoria de estadística o ciencia de datos, seguramente este concepto ya te resulte familiar.

Las estadísticas descriptivas reúnen un conjunto de métodos y herramientas que permiten **resumir y describir** los aspectos más relevantes de un conjunto de datos. Gracias a ellas, es posible obtener una visión más profunda de la información, detectar patrones, identificar valores atípicos y comunicar conclusiones de forma clara y efectiva.

A pesar de su nombre técnico, las estadísticas descriptivas son accesibles y muy útiles para mejorar tus capacidades de análisis. Te invito a adentrarte en este tema y descubrir cómo pueden ayudarte a comprender mejor tus datos.

3.1.- ¿Qué son las estadísticas descriptivas?

Las estadísticas descriptivas constituyen una forma de **sintetizar datos de manera significativa**, permitiendo obtener una visión rápida del conjunto sin necesidad de revisar cada valor individual. Funcionan como una “primera mirada” que facilita entender la estructura general de los datos antes de profundizar en análisis más complejos.

Es como conocer a alguien por primera vez. Obtienes una idea general de quién es a partir de su apariencia, su forma de hablar y algunos datos básicos. Del mismo modo, las estadísticas descriptivas ofrecen una visión general del conjunto de datos, permitiéndote entender sus rasgos principales y tomar decisiones fundamentadas.

Además, las estadísticas descriptivas permiten identificar patrones y relaciones dentro de los datos, lo cual resulta útil para anticipar tendencias o apoyar decisiones informadas. En conjunto, constituyen una herramienta poderosa para interpretar información y forman una parte esencial de cualquier proceso de análisis de datos.

3.2.- Medidas de Tendencia Central

La tendencia central es un concepto estadístico que describe el “centro” de un conjunto de datos, es decir, el punto alrededor del cual se agrupan la mayoría de los valores. Para comprenderla, es fundamental conocer tres medidas principales:

- **Media:** también llamada promedio. Se obtiene sumando todos los valores y dividiéndolos entre la cantidad total de datos. Es útil porque incorpora todos los valores y ofrece un único número representativo del centro.



- **Mediana:** es el valor que ocupa la posición central cuando los datos están ordenados. Si el número de valores es par, la mediana es el promedio de los dos valores centrales. Es especialmente útil porque no se ve tan afectada por valores extremos como la media.
- **Moda:** es el valor o valores que aparecen con mayor frecuencia en el conjunto de datos. Si ningún valor se repite, entonces no existe moda. Es útil para identificar qué valores son más comunes.

En resumen, comprender la tendencia central y estas tres medidas clave ayuda a obtener una visión más clara de la distribución de los datos y aporta información valiosa para análisis posteriores.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({
    'Age': [25, 30, 35, 40, 45, 50, 55, 60, 65, 70]
})

# Calculate mean, median, and mode
mean_age = df['Age'].mean()
median_age = df['Age'].median()
mode_age = df['Age'].mode()

print(f"Mean Age: {mean_age}")
print(f"Median Age: {median_age}")
print(f"Mode Age: {mode_age.tolist()}")
```

3.3.- Medidas de Variabilidad

Para obtener una comprensión más profunda de los datos, puedes explorar diversas medidas de dispersión que te ayuden a comprender cuán dispersos están los datos. Además del rango, que es la diferencia entre los valores máximo y mínimo, existen otras medidas que proporcionan información valiosa.

Una de esas medidas es la **varianza**, que calcula qué tan lejos está cada valor del conjunto de datos respecto a la media. Esta métrica es especialmente útil porque considera todos los valores y cuantifica cuánto varían en relación con la media.

Otra medida estrechamente relacionada es la **desviación estándar**, que es simplemente la raíz cuadrada de la varianza. Esta medida resulta muy útil para obtener una visión más clara de la dispersión de los datos.

Al explorar diferentes medidas de dispersión, puedes obtener una comprensión más completa del conjunto de datos y descubrir patrones o ideas que no son evidentes al observar únicamente los valores en bruto.



Así es como puedes calcular estas medidas:

```
# Calculate range, variance, and standard deviation
range_age = df['Age'].max() - df['Age'].min()
variance_age = df['Age'].var()
std_deviation_age = df['Age'].std()

print(f"Range of Age: {range_age}")
print(f"Variance of Age: {variance_age}")
print(f"Standard Deviation of Age: {std_deviation_age}")
```

3.4.- ¿Por qué es útil?

Las estadísticas descriptivas son una herramienta fundamental en el análisis de datos. Ofrecen un resumen claro y conciso de la información, lo que facilita su interpretación y la extracción de conclusiones. Ya sea analizando el comportamiento de clientes o revisando historiales médicos, estas estadísticas pueden revelar patrones, tendencias y relaciones importantes dentro de los datos.

Aunque Python es una excelente opción para realizar estos cálculos, también existen otras herramientas como Excel, R o software estadístico especializado. Sin embargo, comprender bien los conceptos básicos es esencial para aplicarlos en cualquier entorno y tomar decisiones informadas basadas en los datos. Con este conocimiento, podrás analizar información con seguridad y obtener ideas valiosas que te ayuden a mejorar tus decisiones.

3.5.- Ejemplo: Examinando datos de ventas

Imagina que cuentas con un conjunto de datos que recoge los ingresos mensuales de tu empresa durante el último año. Quieres identificar tendencias centrales y variaciones dentro de esos valores.

Así podrías hacerlo en Python:

```
# Sample sales data for the past 12 months (in 1000s)
sales_data = pd.DataFrame({
    'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],
    'Revenue': [200, 220, 250, 275, 300, 320, 350, 370, 400, 420, 450, 475]
})

# Calculate mean, median, and mode
mean_sales = sales_data['Revenue'].mean()
median_sales = sales_data['Revenue'].median()
mode_sales = sales_data['Revenue'].mode()

print(f"Mean Revenue: {mean_sales}k")
print(f"Median Revenue: {median_sales}k")
print(f"Mode Revenue: {mode_sales.tolist()}k")
```



3.6.- Ejemplo: Examinando reseñas de clientes

Supongamos que estás revisando las opiniones de los clientes en una escala del 1 al 5, y te gustaría saber cómo se distribuyen las calificaciones, cuán variables son y dónde se encuentra la tendencia central.

```
# sample customer review ratings
reviews_data = pd.DataFrame({
    'customerID': range(1, 9),
    'Rating': [5, 4, 5, 2, 5, 3, 2, 1]
})

# Calculate mean, median, and mode
mean_rating = reviews_data['Rating'].mean()
median_rating = reviews_data['Rating'].median()
mode_rating = reviews_data['Rating'].mode()

# Calculate range, variance, and standard deviation
range_rating = reviews_data['Rating'].max() - reviews_data['Rating'].min()
variance_rating = reviews_data['Rating'].var()
std_deviation_rating = reviews_data['Rating'].std()

print(f'Mean Rating: {mean_rating}')
print(f'Median Rating: {median_rating}')
print(f'Mode Rating: {mode_rating.tolist()}')
print(f'Range of Ratings: {range_rating}')
print(f'Variance of Ratings: {variance_rating}')
print(f'Standard Deviation of Ratings: {std_deviation_rating}')
```

Al ejecutar estas simples líneas de código, obtendrás una comprensión completa del conjunto de datos con el que estás trabajando. Este es un importante primer paso para analizar tus datos y obtener ideas valiosas. Las estadísticas descriptivas que se muestran en este código pueden permitirte tomar conjuntos de datos complejos y grandes y simplificarlos en ideas significativas en las que puedes actuar.

De hecho, las estadísticas descriptivas son una herramienta esencial para cualquier analista de datos o investigador. Proporcionan una forma de resumir y comunicar aspectos clave de los datos, como la tendencia central, la variabilidad y la forma de tu conjunto de datos. Al comprender estas características clave de tus datos, puedes comenzar a identificar tendencias y patrones que pueden estar ocultos dentro de los datos.

Entonces, siéntete libre de ajustar los ejemplos de código con tus datos para ver qué tipo de tendencias y patrones surgen. ¡Puede que te sorprendas con lo que descubras! Recuerda que cuanto más explores tus datos utilizando estadísticas descriptivas, más ideas obtendrás y más informadas serán tus decisiones.



3.7.- Asimetría y Curtosis

La **asimetría** es una medida estadística que permite determinar el grado de simetría en una distribución.

Un valor cercano a 0 indica que los datos son aproximadamente simétricos. Si la asimetría es **negativa**, se dice que los datos están *sesgados hacia la izquierda*, lo que significa que la cola izquierda es más larga que la derecha. Si la asimetría es **positiva**, los datos están *sesgados hacia la derecha*, indicando que la cola derecha es más larga que la izquierda.

La **curtosis**, por su parte, mide la “colitud” de la distribución. Un valor de curtosis **mayor que 3** (comparado con una distribución normal) indica la presencia de más valores atípicos: los datos están más concentrados alrededor de la media y menos dispersos en las colas.

Un valor **menor que 3** sugiere menos valores atípicos y una mayor dispersión hacia las colas. La curtosis ayuda a comprender la forma de la distribución y la presencia de valores extremos.

Aquí tienes un ejemplo rápido en Python usando los datos de ventas:

```
# Calculate skewness and kurtosis
skewness = sales_data['Revenue'].skew()
kurtosis = sales_data['Revenue'].kurt()

print(f"Skewness of Revenue: {skewness}")
print(f"Kurtosis of Revenue: {kurtosis}")
```

Incorporar estas métricas puede ofrecer una visión más completa de tus datos y ayudarte a tomar decisiones mejor fundamentadas.

Ejercicios Prácticos para el Capítulo 3.1

Deberás realizar un cuaderno de Jupiter, MUY BIEN EXPLICADO CON COMENTARIOS EN MARKDOWN.

Ejercicio 1: Comprender la Importancia del EDA

Carga un conjunto de datos de tu elección. Realiza exploraciones iniciales como `.head()`, `.info()` y `.describe()` para familiarizarte con su estructura.

Ejercicio 2: Identificar Tipos de Datos

Identifica al menos dos columnas en tu conjunto de datos que contengan datos categóricos y dos que contengan datos numéricos.

Ejercicio 3: Calcular Estadísticas Descriptivas

Calcula la media, mediana y desviación estándar de una columna numérica en tu conjunto de datos.



Ejercicio 4: Comprender la Asimetría y la Kurtosis

Calcula la asimetría y la kurtosis para una columna numérica en tu conjunto de datos.

3.8.- Conclusiones sobre la importancia del EDA

En ESTOS APARTADOS hemos recorrido el mundo del análisis exploratorio de datos (EDA), desde su papel esencial en cualquier proyecto de ciencia de datos hasta las herramientas prácticas que nos ayudan a entender mejor la información que tenemos delante. El EDA es mucho más que una etapa técnica: es el momento en que los datos empiezan a hablar, a mostrar patrones, anomalías y relaciones que nos orientan hacia decisiones más acertadas y modelos más sólidos.

Al principio vimos por qué no conviene saltarse esta fase. Igual que un pintor no empieza a aplicar color sin antes trazar el boceto, un buen analista de datos necesita explorar antes de modelar. Observar los datos con atención desde el inicio nos permite entender qué tenemos entre manos y cómo abordarlo con criterio.

También hablamos sobre los distintos tipos de datos (categóricos y numéricos) y cómo reconocerlos nos ayuda a elegir las técnicas adecuadas. Saber qué tipo de información estamos manejando evita errores y hace que el análisis sea más ágil, sobre todo cuando trabajamos con conjuntos de datos grandes o complejos.

Más adelante, nos adentramos en las estadísticas descriptivas. Vimos cómo medidas como la media, la mediana, la desviación estándar, la asimetría y la curtosis nos permiten resumir grandes volúmenes de datos en cifras que realmente dicen algo. Y lo mejor: aprendimos a calcularlas con Python, lo que nos da herramientas concretas para aplicar todo esto en la práctica.

Finalmente, los ejercicios del capítulo te invitan a poner en marcha lo aprendido. Están pensados para ayudarte a consolidar los conceptos clave y prepararte para retos más avanzados.

Lo que aprendido aquí será una base sólida para todo lo que viene.

CAPÍTULO 4.- PREPROCESAMIENTO DE DATOS

4.1.- Limpieza de datos

La limpieza de datos es un paso fundamental dentro del preprocesamiento y, aun así, suele ser ignorado. Trabajar con datos sin depurar es comparable a intentar pintar sobre un lienzo sucio: cualquier imperfección del soporte afectará al resultado final. Del mismo modo, un análisis basado en datos desordenados puede conducir a conclusiones incorrectas o engañosas.

Por ello, es esencial comprender por qué la limpieza de datos es tan importante y cómo realizarla adecuadamente. Este proceso implica detectar y corregir problemas como valores faltantes, registros duplicados o tipos de datos incorrectos.



En ocasiones también es necesario transformar los datos para que resulten más útiles e interpretables durante el análisis. Además, limpiar datos exige conocer bien el contexto y la naturaleza del conjunto de datos, ya que solo así se garantiza que el resultado final sea fiable y preciso.

En resumen, dedicar tiempo y esfuerzo a la limpieza de datos es imprescindible para asegurar que la información resultante sea de alta calidad y pueda utilizarse con confianza en procesos analíticos y en la toma de decisiones.

(ESTE CAPÍTULO SE HA ESTUDIADO EN LA UNIDAD ANTERIOR PARA TRABAJAR CON LA LIBRERÍA PANDAS)

CAPÍTULO 5: ANÁLISIS EXPLORATORIO DE DATOS VISUAL

A menudo, las visualizaciones son más efectivas para resaltar ideas que los números o tablas en bruto, y es por eso que en este capítulo exploraremos una variedad de técnicas de Visual EDA. Comenzaremos cubriendo el Análisis Univariado, que se centra en analizar una variable a la vez. En esta sección, aprenderemos cómo crear histogramas, gráficos de densidad y diagramas de caja para obtener una comprensión más profunda de nuestros datos.

También discutiremos la importancia de elegir visualizaciones apropiadas para diferentes tipos de datos y exploraremos algunos errores comunes a evitar al trabajar con visualizaciones. Con el Visual EDA, desbloquearemos una herramienta poderosa que nos permite ver nuestros datos de nuevas y emocionantes formas, lo que nos lleva a obtener ideas más significativas y tomar mejores decisiones.

5.1 ANÁLISIS UNIVARIADO

El análisis univariado es una forma crucial de análisis que sirve como un paso fundamental para comprender su conjunto de datos. Involucra examinar una sola variable, lo que puede parecer una tarea simple, pero es un paso importante que proporciona información valiosa sobre la naturaleza de sus datos. Al visualizar variables individuales, puede obtener una mejor comprensión de su distribución, tendencias y peculiaridades.

Este proceso puede ayudarlo a identificar tendencias y patrones que pueden estar presentes en sus datos, lo que le permite tomar decisiones informadas basadas en sus hallazgos. Por ejemplo, si está examinando un conjunto de datos de compras de clientes, el análisis univariado puede ayudarlo a comprender los productos más populares, la frecuencia de las compras y el monto promedio gastado por transacción.

Esto puede ayudarlo a adaptar sus estrategias de marketing y ventas para satisfacer mejor las necesidades y preferencias de sus clientes. Por lo tanto, es esencial llevar a cabo un análisis univariado como parte de su proceso de análisis de datos para obtener una comprensión integral de sus datos.



5.1.1 Histogramas

Los histogramas son una herramienta esencial en el análisis de datos univariados. Nos proporcionan una representación visual de la distribución de una variable numérica, lo que nos permite evaluar la forma de los datos e identificar patrones o tendencias. Al examinar el histograma, podemos obtener información sobre la tendencia central de los datos, así como su variabilidad y dispersión.

Además, los histogramas son altamente personalizables y se pueden utilizar para explorar una amplia gama de tipos de datos y variables, lo que los convierte en una herramienta versátil y valiosa para cualquier analista de datos o investigador. En general, los histogramas son una piedra angular del análisis exploratorio de datos y una técnica fundamental para obtener una comprensión más profunda de nuestros datos.

Así es cómo trazar un histograma usando Matplotlib:

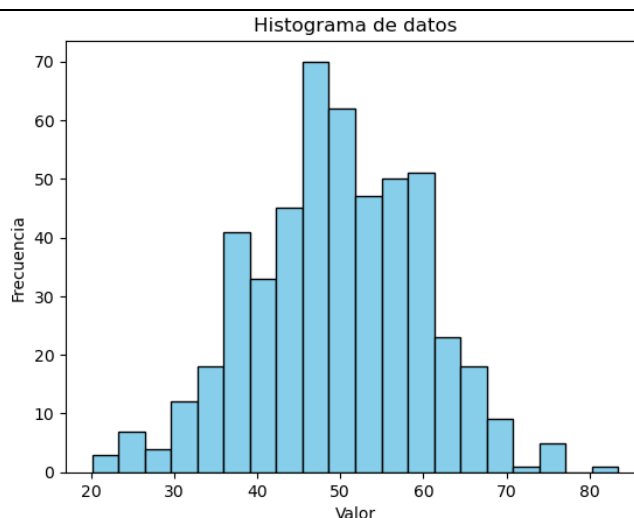
```
import matplotlib.pyplot as plt
import numpy as np

# Generamos datos de ejemplo
datos = np.random.normal(loc=50, scale=10, size=500)

# Creamos el histograma
plt.hist(datos, bins=20, color='skyblue', edgecolor='black')

# Añadimos título y etiquetas
plt.title("Histograma de datos")
plt.xlabel("Valor")
plt.ylabel("Frecuencia")

# Mostramos la gráfica
plt.show()
```



Desde la perspectiva del **análisis exploratorio de datos (EDA)**, este histograma ya te ofrece varias pistas importantes sobre la estructura de la variable que estás estudiando. Te lo desgloso como lo trabajarías en clase con tu alumnado:

1. Forma de la distribución

La curva que dibujan las barras sugiere una **distribución aproximadamente normal**:

- simétrica,
- con mayor concentración en el centro,
- y menos frecuencia en los extremos.

Esto indica que la mayoría de los valores se agrupan alrededor de un punto central.

2. Tendencia central



El pico del histograma está alrededor de **50**, lo que sugiere que:

- la **media** y la **mediana** probablemente estén cerca de ese valor,
- es el rango donde se concentran más observaciones.

3. Dispersión

El rango visible va aproximadamente de **20 a 80**, lo que te da una idea de la **variabilidad** de la variable.

La anchura de la distribución indica que los datos no están excesivamente concentrados ni extremadamente dispersos.

4. Ausencia de valores atípicos evidentes

No se observan barras aisladas en los extremos. Esto sugiere que **no hay outliers fuertes** o, al menos, no son visualmente prominentes.

5. Tamaño de muestra razonable

La suavidad de la forma indica que hay suficientes datos como para que el histograma sea estable y representativo.

6. Primeras hipótesis para el análisis

Con esta distribución podrías plantear:

- que la variable sigue un comportamiento natural o esperado (según una db. Normal),
- que no hay sesgos fuertes,
- que es razonable aplicar técnicas estadísticas que asumen normalidad (siempre verificándolo con más pruebas).

5.1.2 Diagramas de Caja

Los diagramas de caja, también conocidos como diagramas de caja y bigotes, son un tipo de representación gráfica que se utiliza con frecuencia en el análisis estadístico. Son particularmente útiles para representar la dispersión de una variable, proporcionando un resumen visual de su distribución. Los diagramas de caja se construyen utilizando cinco medidas estadísticas: el mínimo, el primer cuartil, la mediana, el tercer cuartil y el máximo.

El cuadro en el diagrama representa el rango intercuartílico (IQR), que es la distancia entre el primer y tercer cuartil. Los bigotes se extienden desde el cuadro hasta los valores mínimo y máximo, excluyendo cualquier punto de datos identificado como valores atípicos. El uso de diagramas de caja permite una forma rápida y fácil de comparar la dispersión de diferentes variables e identificar posibles valores atípicos en los datos.



Aquí tienes un simple diagrama de caja usando Matplotlib:

```
import matplotlib.pyplot as plt
import numpy as np

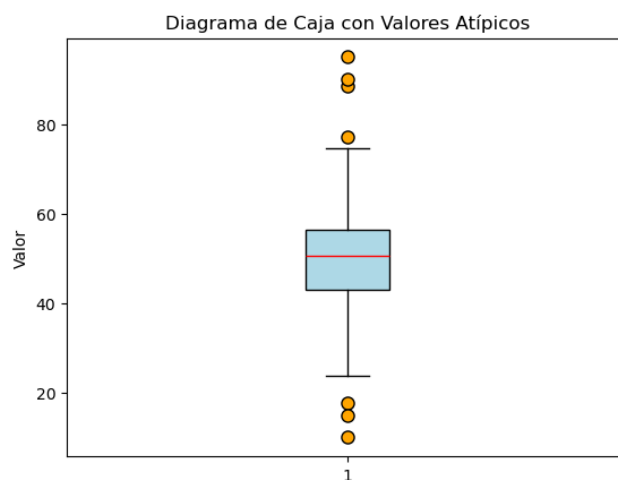
# Generamos datos con valores extremos
np.random.seed(42)
datos = np.random.normal(loc=50, scale=10, size=300)

# Añadimos valores atípicos manualmente
outliers = np.array([10, 15, 90, 95])
datos_con_outliers = np.concatenate([datos, outliers])

# Dibujamos el diagrama de caja
plt.boxplot(datos_con_outliers, vert=True, patch_artist=True,
            boxprops=dict(facecolor='lightblue', color='black'),
            medianprops=dict(color='red'),
            whiskerprops=dict(color='black'),
            capprops=dict(color='black'),
            flierprops=dict(marker='o', markerfacecolor='orange', markersize=8))

plt.title("Diagrama de Caja con Valores Atípicos")
plt.ylabel("Valor")

plt.show()
```



Interpretación del diagrama de caja desde el EDA

El diagrama de caja nos permite entender rápidamente cómo se distribuye esta variable y detectar posibles problemas o patrones interesantes en los datos.

1. Tendencia central

La línea roja dentro de la caja representa la mediana, que resume el valor central de la distribución. En este caso, la mediana está aproximadamente en el centro de la caja, lo que indica que la variable no presenta un sesgo fuerte hacia valores altos o bajos.

2. Dispersión

La caja representa el rango intercuartílico (IQR), es decir, el 50% central de los datos. Su tamaño moderado sugiere que la mayor parte de los valores están relativamente concentrados. Los bigotes muestran hasta dónde llegan los valores que se consideran “normales” según la regla de 1.5·IQR.

3. Valores atípicos

Los puntos naranjas fuera de los bigotes son valores atípicos. Su presencia indica que existen observaciones que se alejan significativamente del comportamiento general. Desde el EDA, esto nos lleva a plantear preguntas clave:

- ¿Son errores de registro o medición?
- ¿Representan casos reales pero poco frecuentes?
- ¿Forman un subgrupo que merece análisis separado?

Los outliers no deben eliminarse automáticamente: primero hay que entender su origen.

4. Simetría



La posición de la mediana y la longitud similar de los bigotes sugieren que la distribución es aproximadamente simétrica. Esto facilita el uso de medidas como la media y permite aplicar técnicas estadísticas que asumen cierta normalidad.

5. Calidad del dato

La combinación de:

- caja compacta,
- bigotes razonables,
- y outliers aislados, indica que los datos son estables, pero contienen algunos valores extremos que deben analizarse con cuidado.

6. Próximos pasos del EDA

A partir de este boxplot, sería razonable:

- comparar esta variable con otras,
- estudiar si los outliers afectan a la media,
- revisar el proceso de recogida de datos,
- segmentar la población para ver si los valores extremos pertenecen a un grupo concreto.

5.1.3 Gráficos de conteo para datos categóricos

Cuando se trata de datos categóricos, un gráfico de conteo puede ser una herramienta extremadamente informativa para usar en el análisis de datos. En un gráfico de conteo, cada categoría se representa por una barra cuya altura corresponde a la frecuencia de esa categoría en el conjunto de datos. Al examinar la altura de cada barra, podemos determinar rápidamente qué categorías son más comunes y cuáles son menos frecuentes.

Esta información se puede utilizar para identificar patrones o tendencias en los datos que pueden no ser evidentes desde una simple inspección de los datos en bruto. Además, los gráficos de conteo se pueden usar para comparar la frecuencia de las categorías en diferentes subgrupos de datos. Por ejemplo, podríamos crear un gráfico de conteo que muestre la frecuencia de cada categoría desglosada por género o grupo de edad.

Esto puede ayudarnos a identificar cualquier diferencia o similitud en la forma en que diferentes subgrupos de datos se distribuyen entre las categorías. Por lo tanto, los gráficos de conteo son una herramienta valiosa en el arsenal del analista de datos, proporcionando una forma simple pero poderosa de explorar y visualizar datos categóricos.

Así es cómo puedes hacer uno usando Seaborn:

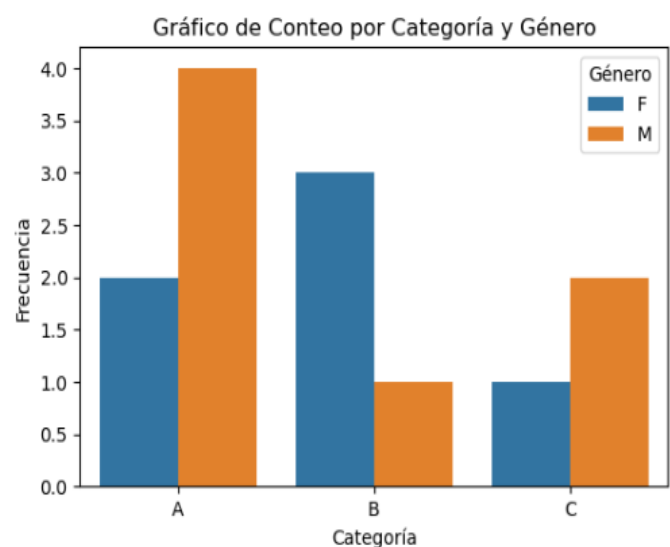
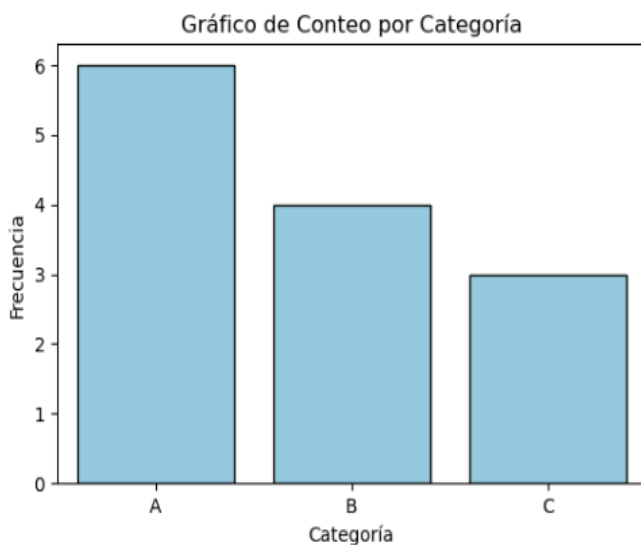
```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
# Creamos un conjunto de datos de ejemplo
datos = pd.DataFrame({
    "Categoría": ["A","A","A", "B", "C", "A", "B", "B","A", "C", "C", "B", "A"],
    "Género": ["F", "F", "M", "F", "M", "M", "F", "F", "M", "F","M", "M", "M"]
})
```

```
# Gráfico de conteo simple
```



```
plt.figure(figsize=(6,4))
sns.countplot(data=datos, x="Categoría", color="skyblue", edgecolor="black")
plt.title("Gráfico de Conteo por Categoría")
plt.xlabel("Categoría")
plt.ylabel("Frecuencia")
plt.show()

# Gráfico de conteo desglosado por subgrupo (ej. género)
plt.figure(figsize=(6,4))
sns.countplot(data=datos, x="Categoría", hue="Género")
plt.title("Gráfico de Conteo por Categoría y Género")
plt.xlabel("Categoría")
plt.ylabel("Frecuencia")
plt.show()
```



Interpretación de los Gráficos de Conteo desde el EDA

A continuación, se muestran dos gráficos de conteo que permiten analizar datos categóricos y su distribución dentro del conjunto de datos. Ambos gráficos son herramientas fundamentales dentro del **Análisis Exploratorio de Datos (EDA)**, ya que ayudan a identificar patrones, desequilibrios y relaciones entre variables.

1. Gráfico de Conteo por Categoría

Este gráfico muestra cuántas veces aparece cada categoría (A, B y C) en el conjunto de datos.

Interpretación

- La categoría **A** es la más frecuente, con 6 apariciones.
- La categoría **B** aparece 4 veces.
- La categoría **C** es la menos frecuente, con 3 apariciones.

Qué aporta en EDA

- Permite detectar **desbalances** entre categorías (A domina claramente).
- Ayuda a comprender la **estructura general** del dataset antes de aplicar análisis más complejos.



- Facilita la identificación de categorías minoritarias que podrían requerir un tratamiento especial en fases posteriores del análisis.

2. Gráfico de Conteo por Categoría y Género

Este gráfico muestra la frecuencia de cada categoría, pero desglosada por género (F y M). Esto permite analizar cómo se distribuyen las categorías dentro de un subgrupo.

Interpretación

- En la categoría **A**, predominan los hombres (4) frente a las mujeres (2).
- En la categoría **B**, ocurre lo contrario: hay más mujeres (3) que hombres (1).
- En la categoría **C**, la distribución es más equilibrada: 1 mujer y 2 hombres.

Qué aporta en EDA

- Permite identificar **diferencias entre subgrupos** dentro de cada categoría.
- Ayuda a detectar **posibles sesgos** en la distribución de los datos.
- Facilita la formulación de preguntas clave, como:
 - ¿Por qué un género aparece más en una categoría concreta?
 - ¿Existe algún patrón que deba investigarse más a fondo?
 - ¿Es necesario equilibrar los datos antes de modelizar?

Conclusión General

Los gráficos de conteo permiten:

- visualizar rápidamente la distribución de categorías,
- detectar desequilibrios,
- comparar subgrupos,
- y comprender mejor la estructura del conjunto de datos.

Son una herramienta esencial en el EDA, ya que proporcionan una visión clara y directa de cómo se organizan los datos categóricos y permiten tomar decisiones informadas antes de avanzar hacia análisis más profundos.

5.1.4 Estadísticas descriptivas junto con Visualizaciones

Los gráficos proporcionan una excelente manera de representar visualmente los datos, pero también es esencial considerar los valores numéricos porque ofrecen una perspectiva diferente. Combinar tanto los datos visuales como numéricos puede proporcionar una comprensión más completa del conjunto de datos.

En el ejemplo a continuación, tenemos un código que crea un histograma para un conjunto de datos aleatorio y calcula su media y desviación estándar. Analizar el histograma nos proporciona una representación visual de la distribución de los datos, mientras que la media y la desviación estándar nos proporcionan valores numéricos que describen la tendencia central y la variabilidad del conjunto de datos. Al analizar tanto los datos visuales como numéricos

```
import matplotlib.pyplot as plt
import numpy as np
```



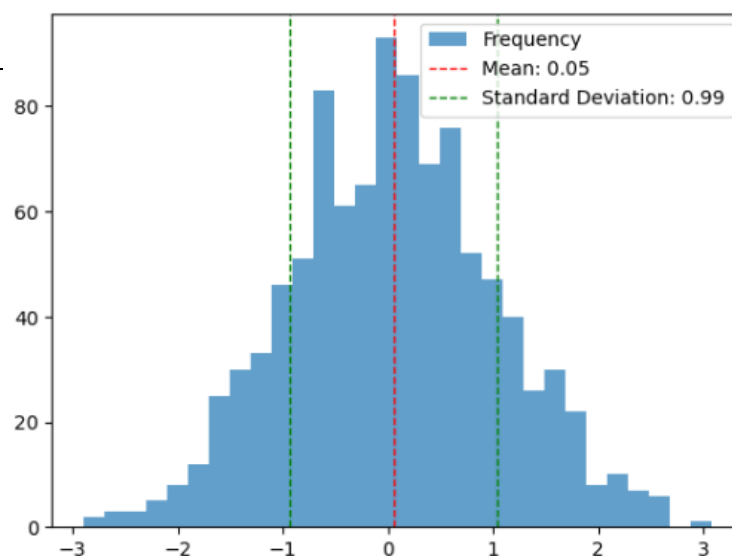
```
# Generate a random dataset
data = np.random.normal(0, 1, 1000)

# Calculate mean and standard deviation
mean_value = np.mean(data)
std_value = np.std(data)

# Create the histogram
plt.hist(data, bins=30, alpha=0.7, label='Frequency')

# Mark mean and standard deviation
plt.axvline(mean_value, color='r', linestyle='dashed', linewidth=1, label=f'Mean: {mean_value:.2f}')
plt.axvline(mean_value + std_value, color='g', linestyle='dashed', linewidth=1, label=f'Standard Deviation: {std_value:.2f}')
plt.axvline(mean_value - std_value, color='g', linestyle='dashed', linewidth=1)

plt.legend()
plt.show()
```



Interpretación del Histograma con Media y Desviación Estándar (EDA)

Este histograma muestra la distribución de un conjunto de datos numéricos y añade información estadística clave: la **media** (línea roja discontinua) y la **desviación estándar** (líneas verdes discontinuas). Esta combinación de visualización y estadísticos descriptivos permite obtener una comprensión completa del comportamiento de la variable.

1. Forma de la distribución

La forma del histograma es aproximadamente **simétrica y con apariencia de campana**, lo que sugiere que los datos siguen una distribución cercana a la normal.

Esto indica que la mayoría de los valores se concentran alrededor del centro y disminuyen progresivamente hacia los extremos.

2. Tendencia central: la media

La línea roja marca la **media**, situada alrededor de 0.05.



Esto nos dice que el valor promedio del conjunto de datos está muy cerca de cero, lo cual es coherente con una distribución normal centrada.

La media coincide visualmente con el punto donde el histograma alcanza su mayor altura, lo que refuerza la idea de simetría.

3. Variabilidad: la desviación estándar

Las líneas verdes representan **una desviación estándar por encima y por debajo de la media**. En este caso, la desviación estándar es aproximadamente 0.99.

Esto significa que:

- La mayoría de los datos se encuentran entre **media - 1 σ** y **media + 1 σ** , es decir, entre aproximadamente -0.94 y 1.04.
- Este rango contiene la parte más densa del histograma, lo cual es típico de una distribución normal.

La distancia entre estas líneas nos indica cuánta dispersión hay en los datos: una desviación estándar cercana a 1 implica una variabilidad moderada.

4. Detección de valores extremos

No se observan barras aisladas lejos de las líneas verdes, lo que sugiere que **no hay valores atípicos evidentes**.

La distribución es compacta y no presenta colas largas ni picos extraños.

5. Conclusiones desde el EDA

A partir de este histograma, podemos afirmar que la variable analizada presenta un comportamiento **estable, simétrico y sin anomalías visibles**. La media se sitúa prácticamente en el centro de la distribución, lo que indica que no hay desplazamientos significativos hacia valores altos o bajos. La desviación estándar, representada por las líneas verdes, muestra que la mayor parte de los datos se concentra dentro del rango esperado, lo que confirma una variabilidad moderada y coherente.

Además, la ausencia de barras aisladas o colas pronunciadas sugiere que **no existen valores atípicos importantes** ni problemas evidentes en la calidad del dato. En conjunto, esta visualización permite concluir que la distribución es adecuada para aplicar técnicas estadísticas que asumen normalidad y que los datos reflejan un comportamiento regular y consistente.

5.1.5 Gráfico de Densidad de Kernel

El Gráfico de Densidad de Kernel es una herramienta útil para la visualización de datos cuando se desea una representación más suave. En lugar del aspecto de bloques de los histogramas, el Gráfico



de Densidad de Kernel proporciona una curva suave que puede hacer que los patrones y tendencias en los datos sean más evidentes.

Este tipo de gráfico es particularmente útil para conjuntos de datos grandes, ya que la curva suave puede ayudar a identificar picos y valles más pequeños que podrían perderse en un histograma. Además, el Gráfico de Densidad de Kernel se puede utilizar para estimar la función de densidad de probabilidad de una variable, lo que puede proporcionar ideas valiosas sobre la distribución de los datos.

Por lo tanto, es una herramienta valiosa para analistas de datos e investigadores que desean obtener una comprensión más profunda de sus conjuntos de datos.

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Generamos un conjunto de datos con forma aproximadamente normal
np.random.seed(42)
datos = np.random.normal(loc=50, scale=10, size=500)

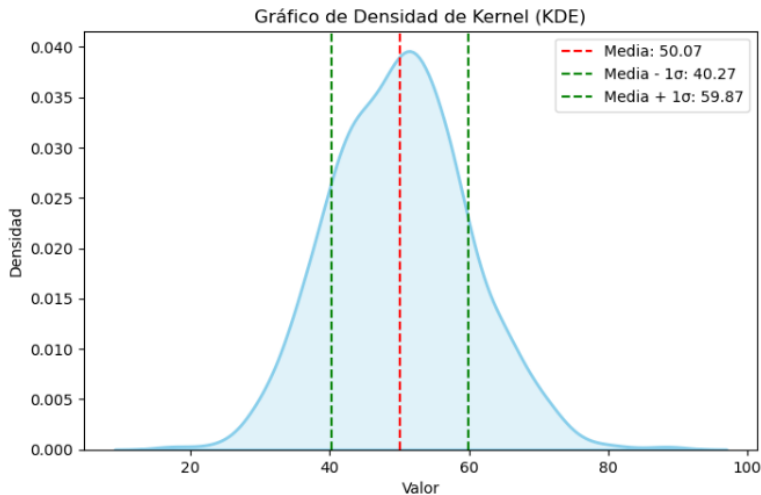
# Creamos el gráfico de densidad de kernel
plt.figure(figsize=(8,5))
sns.kdeplot(datos, fill=True, color="skyblue", linewidth=2)

# Añadimos líneas de media y desviación estándar
media = np.mean(datos)
desv = np.std(datos)

plt.axvline(media, color='red', linestyle='--', label=f"Media: {media:.2f}")
plt.axvline(media - desv, color='green', linestyle='--', label=f"Media - 1σ: {media-desv:.2f}")
plt.axvline(media + desv, color='green', linestyle='--', label=f"Media + 1σ: {media+desv:.2f}")

plt.title("Gráfico de Densidad de Kernel (KDE)")
plt.xlabel("Valor")
plt.ylabel("Densidad")
plt.legend()
plt.show()
```





Conclusiones desde el EDA a partir del Gráfico de Densidad de Kernel (KDE)

El gráfico de Densidad de Kernel permite observar la forma de la distribución de los datos de manera suave y continua. A diferencia del histograma, que muestra bloques, este tipo de visualización facilita la identificación de patrones, picos y tendencias que pueden pasar desapercibidos en otras representaciones.

A partir del KDE mostrado, podemos extraer las siguientes conclusiones:

1. La distribución es aproximadamente normal

La curva presenta un único pico central y una forma simétrica. Esto indica que los datos siguen un patrón cercano a la **distribución normal**, donde la mayoría de los valores se concentran alrededor del centro y disminuyen progresivamente hacia los extremos.

2. La media representa bien el centro de los datos

La línea roja discontinua marca la **media**, situada alrededor de 50. La curva alcanza su punto más alto cerca de esa zona, lo que confirma que la media es un buen indicador de la tendencia central.

3. La dispersión es moderada y coherente

Las líneas verdes representan **una desviación estándar por encima y por debajo de la media**. La mayor parte de la densidad se encuentra dentro de ese intervalo, lo que indica que los datos están razonablemente concentrados alrededor del valor central.

Esto sugiere una variabilidad **estable y sin extremos pronunciados**.

4. No se observan colas largas ni asimetrías



La curva cae de forma similar hacia ambos lados, lo que indica ausencia de sesgo. Tampoco se aprecian colas alargadas, que serían indicio de valores extremos o comportamientos anómalos.

5. No hay señales de valores atípicos importantes

Aunque el KDE no muestra puntos individuales, sí permite detectar irregularidades en la forma de la distribución. En este caso, la curva es suave y regular, sin picos secundarios ni deformaciones, lo que sugiere que **no existen outliers significativos**.

6. Conclusión general desde el EDA

El análisis del KDE permite concluir que:

- La variable presenta un comportamiento **estable, simétrico y sin anomalías visibles**.
- La media y la desviación estándar describen adecuadamente la distribución.
- Los datos parecen limpios, homogéneos y adecuados para aplicar técnicas estadísticas que asumen normalidad.
- No se detectan patrones extraños, subgrupos ocultos ni valores extremos que requieran tratamiento especial.

5.1.6 Diagrama de Violín

Un **diagrama de violín** es una visualización que combina dos elementos:

- La forma del **diagrama de densidad** (similar a un KDE), que muestra cómo se distribuyen los valores.
- Un **boxplot** incrustado en el centro (mediana, cuartiles), aunque a veces no aparece explícito.

Su nombre viene de que la forma resultante recuerda a un violín.

Para qué sirve en EDA

- Permite ver **la distribución completa** de una variable numérica: zonas con más densidad, colas, multimodalidad (varios picos).
- Es más informativo que un boxplot cuando te interesa la **forma** de la distribución, no solo sus resúmenes.
- Facilita comparar distribuciones entre **categorías** (por ejemplo, ingresos por género, notas por grupo, tiempos por tipo de máquina).

Cuando usarlo



- Cuando sospechas que la variable **no es normal**.
- Cuando quieres detectar **asimetrías, picos múltiples, o colas largas**.
- Cuando necesitas comparar grupos con **muestras medianas o grandes**.

5.1.7 Sesgo y Curtosis de los Datos

Son dos medidas que describen la **forma** de una distribución más allá de la media y la desviación típica.

Sesgo (Skewness)

El **sesgo** indica si la distribución está “inclinada” hacia un lado.

- **Sesgo = 0** → distribución simétrica (como una normal ideal).
- **Sesgo > 0** → cola hacia la derecha (valores extremos altos).
- **Sesgo < 0** → cola hacia la izquierda (valores extremos bajos).

Interpretación en EDA

- Un sesgo fuerte sugiere que la media **no es representativa**.
- Puede indicar necesidad de **transformaciones** (log, raíz, Box–Cox).
- Ayuda a detectar **outliers** o procesos generadores no simétricos (ingresos, tiempos de espera, ventas).

Curtosis (Kurtosis)

La **curtosis** mide cuánta “concentración” hay en el centro y en las colas.

- **Curtosis ≈ 3** → distribución mesocúrtica (similar a la normal).
- **Curtosis > 3** → leptocúrtica:
 - pico más alto
 - colas más pesadas
 - más probabilidad de valores extremos
- **Curtosis < 3** → platicúrtica:
 - pico más bajo
 - colas más ligeras
 - menos valores extremos



Interpretación en EDA

- Curtosis alta → riesgo de **outliers** y colas pesadas.
- Curtosis baja → datos más “planos”, menos extremos.
- Es útil para evaluar si los supuestos de normalidad son razonables.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import skew, kurtosis

# -----
# 1. Creamos un dataset de ejemplo
# -----
np.random.seed(42)

# Distribución con sesgo positivo (cola a la derecha)
data = np.random.exponential(scale=2, size=500)

df = pd.DataFrame({"valor": data})

# -----
# 2. Diagrama de violín
# -----
plt.figure(figsize=(8, 5))
sns.violinplot(x=df["valor"], inner="box", color="skyblue")
plt.title("Diagrama de Violín de la Variable 'valor'")
plt.xlabel("valor")
plt.show()

# -----
# 3. Cálculo de sesgo y curtosis
# -----
sesgo = skew(df["valor"])
curtosis_val = kurtosis(df["valor"], fisher=False) # fisher=False → curtosis clásica (normal = 3)

print(f"Sesgo (skewness): {sesgo:.3f}")
print(f"Curtosis: {curtosis_val:.3f}")

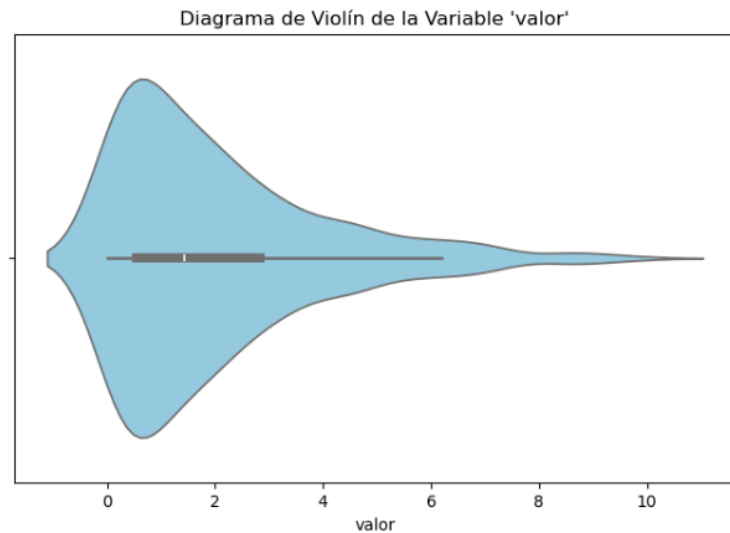
# -----
# 4. Interpretación automática
# -----
def interpretar_sesgo(s):
    if abs(s) < 0.1:
        return "La distribución es prácticamente simétrica."
    elif s > 0:
        return "La distribución tiene sesgo positivo: cola hacia la derecha."
    else:
        return "La distribución tiene sesgo negativo: cola hacia la izquierda."

def interpretar_curtosis(k):
```



```
if abs(k - 3) < 0.1:
    return "Curtosis cercana a 3: similar a una distribución normal."
elif k > 3:
    return "Curtosis alta (leptocúrtica): colas pesadas y más valores extremos."
else:
    return "Curtosis baja (platicúrtica): distribución más plana y con menos valores extremos."

print("\nInterpretación del sesgo:", interpretar_sesgo(sesgo))
print("Interpretación de la curtosis:", interpretar_curtosis(curtosis_val))
```



Sesgo (skewness): 1.441
Curtosis: 4.854

El violín muestra claramente una distribución asimétrica hacia la derecha:

La parte derecha del violín es más larga y extendida, lo que indica una cola derecha pronunciada.

La mayor densidad de valores está cerca del extremo izquierdo, donde el violín es más ancho.

El boxplot interno confirma que la mediana está muy cerca del extremo inferior, típica de distribuciones sesgadas positivamente.

En conjunto, la forma del violín ya anticipa que la variable no es simétrica y que hay valores altos poco frecuentes pero muy alejados.

5.2 ANÁLISIS BIVARIADO

Para tener una buena comprensión del análisis univariado, que se centra en el estudio de una sola variable, es hora de adentrarse en el mundo del análisis bivariado. Este tipo de análisis se enfoca en la relación entre dos variables y cómo se afectan entre ellas.

El análisis bivariado es clave, ya que te permite identificar patrones, tendencias y correlaciones entre variables. Es un eslabón fundamental, ya que pocas veces los fenómenos del mundo real se comportan de manera aislada. Las variables suelen estar interconectadas y es importante poder analizar sus interacciones y relaciones entre ellas, lo que permite una mejor comprensión de los datos y una narrativa más completa.



5.2.1 Gráficos de Dispersión

El gráfico de dispersión es una herramienta increíblemente útil en tu arsenal de visualización de datos. Te permite “ver” visualmente la relación entre dos variables de manera clara y directa. Al observar cómo los datos se extienden en un gráfico de dispersión proporciona una idea de patrones, correlaciones y tendencias.

Los gráficos de dispersión se pueden utilizar para una amplia gama de aplicaciones, desde la ciencia hasta los negocios. En ciencia de datos, por ejemplo, los gráficos de dispersión pueden ayudarte a identificar patrones entre diferentes características de los datos. En negocios, por ejemplo, combinar el uso de análisis visualización de datos puede ayudarte a comprender mejor el comportamiento del cliente.

Ejemplo con Matplotlib:

```
import matplotlib.pyplot as plt
import matplotlib.pyplot as plt
import numpy as np # Librería fundamental para generar datos aleatorios [1]

# 1. Definir la cantidad de puntos
n_puntos = 200

# 2. Generar valores aleatorios para X
# Generamos 200 valores entre 0 y 50
x = np.random.uniform(0, 50, n_puntos)

# 3. Generar valores para Y con pendiente positiva y "ruido"
# Usamos la fórmula Y = pendiente * X + intercepto + ruido
# Una pendiente de 1.5 asegura la relación positiva [4, 5]
pendiente = 1.5
intercepto = 10
ruido = np.random.normal(0, 8, n_puntos) # El ruido hace que los puntos no sean una línea perfecta [6]

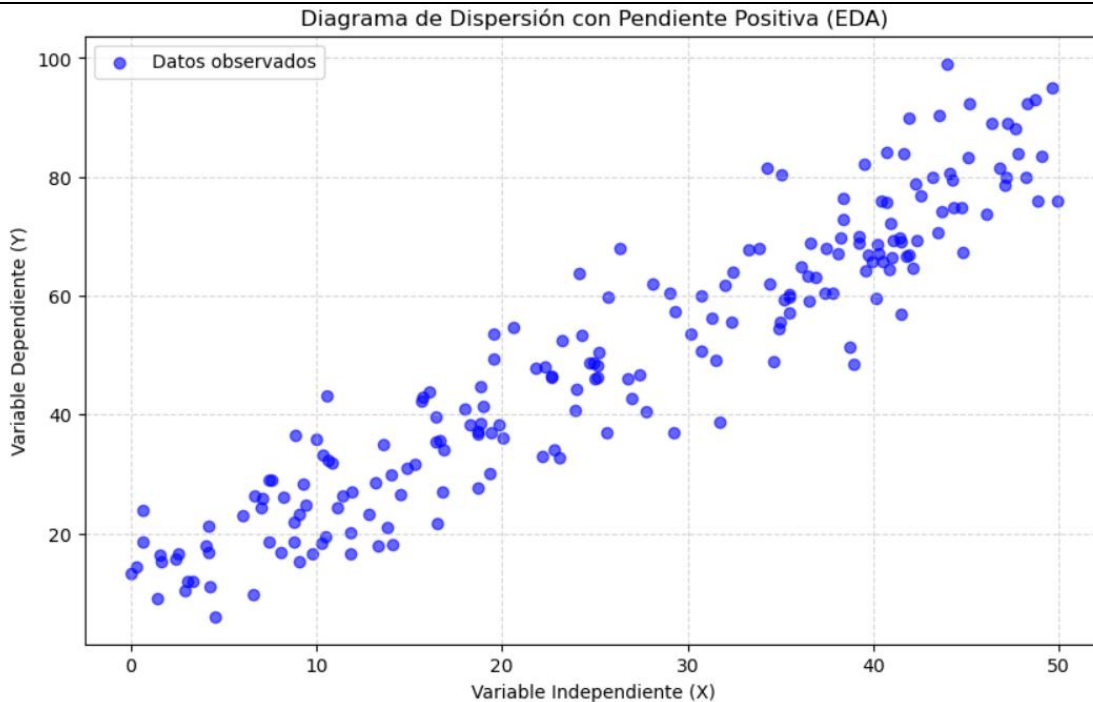
y = pendiente * x + intercepto + ruido

# 4. Crear la visualización [7, 8]
plt.figure(figsize=(10, 6))
plt.scatter(x, y, alpha=0.6, color='blue', label='Datos observados')

# Añadir elementos informativos [2]
plt.xlabel('Variable Independiente (X)')
plt.ylabel('Variable Dependiente (Y)')
plt.title('Diagrama de Dispersión con Pendiente Positiva (EDA)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
```



`plt.show()`



5.2.2 Coeficiente de Correlación

El coeficiente de correlación mide la fuerza y dirección de la relación entre dos variables. Se representa comúnmente con la letra “r” y su valor varía entre -1 y 1.

- $r = 1$: correlación positiva perfecta
- $r = -1$: correlación negativa perfecta
- $r = 0$: sin correlación

Una correlación positiva indica que a medida que una variable aumenta, la otra también lo hace. Una correlación negativa indica que a medida que una variable aumenta, la otra disminuye. Un valor cercano a 0 indica que no hay una relación lineal clara entre las variables

```
import pandas as pd

data = {'x': [1, 2, 3, 4, 5], 'y': [2, 4, 6, 8, 10]}
df = pd.DataFrame(data)

correlation = df['x'].corr(df['y'])
print("Coeficiente de correlación:", correlation)
```

Coeficiente de correlación: 0.9999999999999999

5.2.3 Gráficos de Líneas

Los gráficos de líneas son útiles para visualizar relaciones entre variables a lo largo del tiempo o en secuencia. Permiten observar tendencias, patrones y cambios.



```
import matplotlib.pyplot as plt
import numpy as np

# 1. Configuración de datos
n_puntos = 200
np.random.seed(42) # Para que los resultados sean replicables

# 2. Generar valores aleatorios para la dispersión (Scatter)
x = np.random.uniform(0, 50, n_puntos)
pendiente = 1.5
intercepto = 10
ruido = np.random.normal(0, 8, n_puntos)
y = pendiente * x + intercepto + ruido

# 3. Generar valores para la línea de tendencia (Line Plot)
# Ordenamos x para que la línea se dibuje de forma continua de izquierda a derecha [1]
x_linea = np.sort(x)
y_linea = pendiente * x_linea + intercepto

# 4. Crear la visualización combinada
plt.figure(figsize=(10, 6))

# Dibujo de dispersión: Identifica patrones y correlación [1, 3]
plt.scatter(x, y, alpha=0.5, color='blue', label='Datos con ruido (Dispersión)')

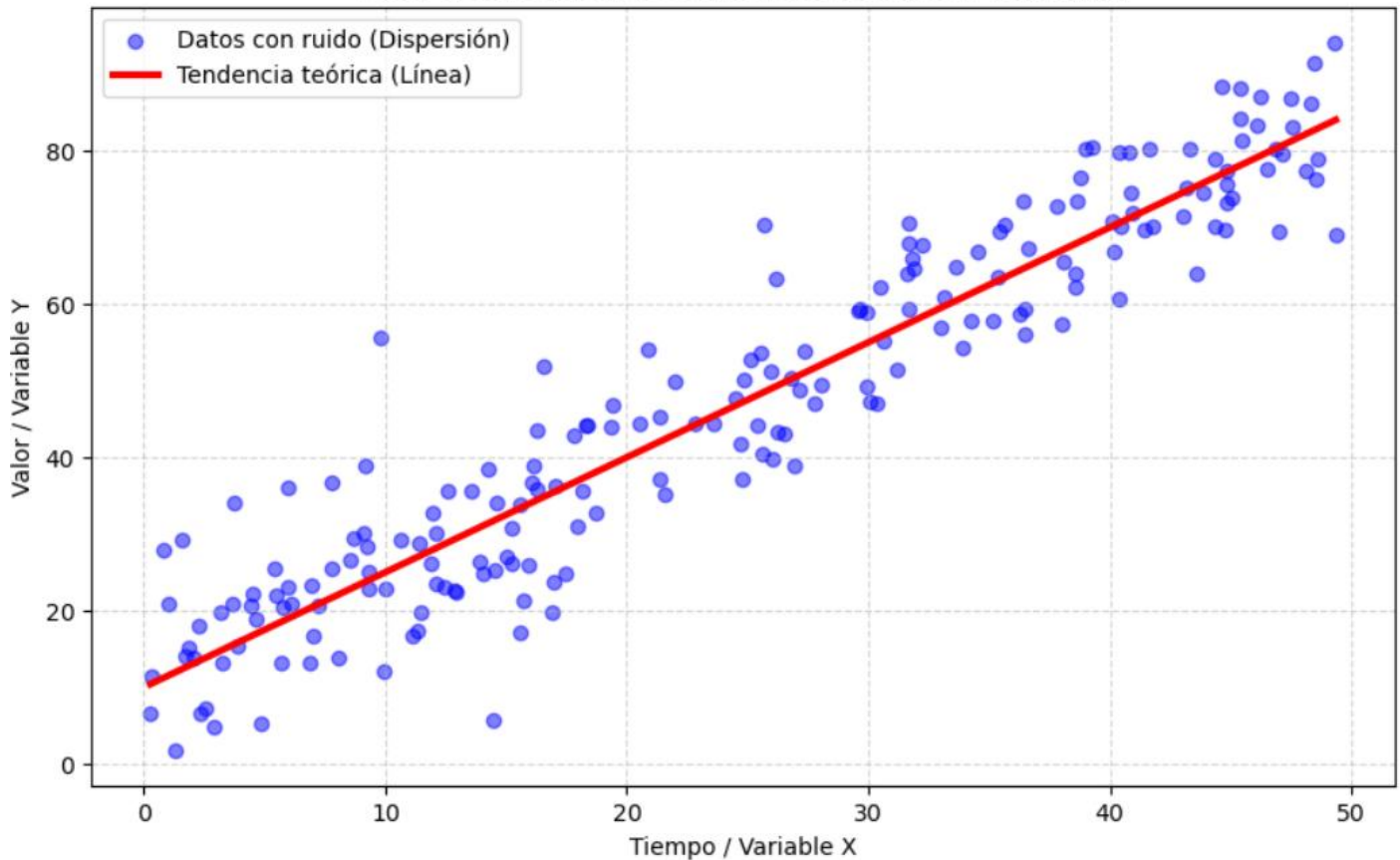
# Dibujo de línea: Resalta la tendencia o evolución [1, 3]
plt.plot(x_linea, y_linea, color='red', linewidth=3, label='Tendencia teórica (Línea)')

# Añadir elementos informativos para mejorar la inteligibilidad [4]
plt.xlabel('Tiempo / Variable X')
plt.ylabel('Valor / Variable Y')
plt.title('EDA: Visualización de Correlación Positiva y Tendencia')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend() # Muestra las etiquetas de los gráficos

plt.show()
```



EDA: Visualización de Correlación Positiva y Tendencia



5.2.4 Mapas de Calor

Los mapas de calor utilizan colores para representar diferentes valores, facilitando la identificación de patrones, correlaciones y tendencias. Son útiles en disciplinas como deportes, geografía, medicina, finanzas, etc.

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

n = 200

# Edad independiente (no influye en ingresos, gasto ni ahorro)
edad = np.random.randint(20, 65, n)

# Ingresos con variabilidad realista
ingresos = np.random.normal(50000, 8000, n)

# Gasto inversamente proporcional a ingresos (más ingresos → menos gasto)
```

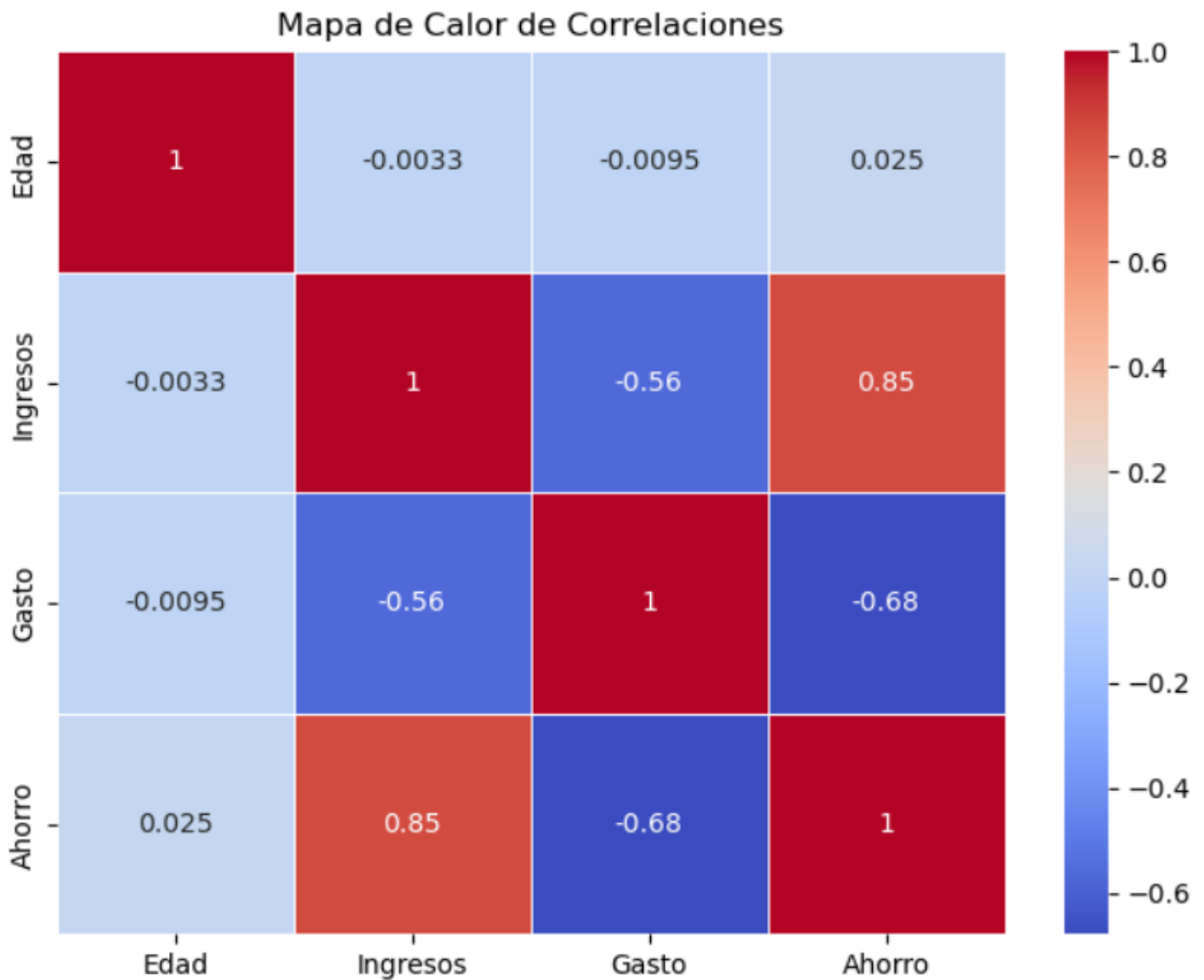


```
gasto = 40000 - ingresos * 0.3 + np.random.normal(0, 3000, n)

# Ahorro inversamente proporcional al gasto y relacionado con ingresos
ahorro = ingresos * 0.4 - gasto * 0.5 + np.random.normal(0, 2000, n)

df = pd.DataFrame({
    'Edad': edad,
    'Ingresos': ingresos,
    'Gasto': gasto,
    'Ahorro': ahorro
})

plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Mapa de Calor de Correlaciones')
plt.show()
```



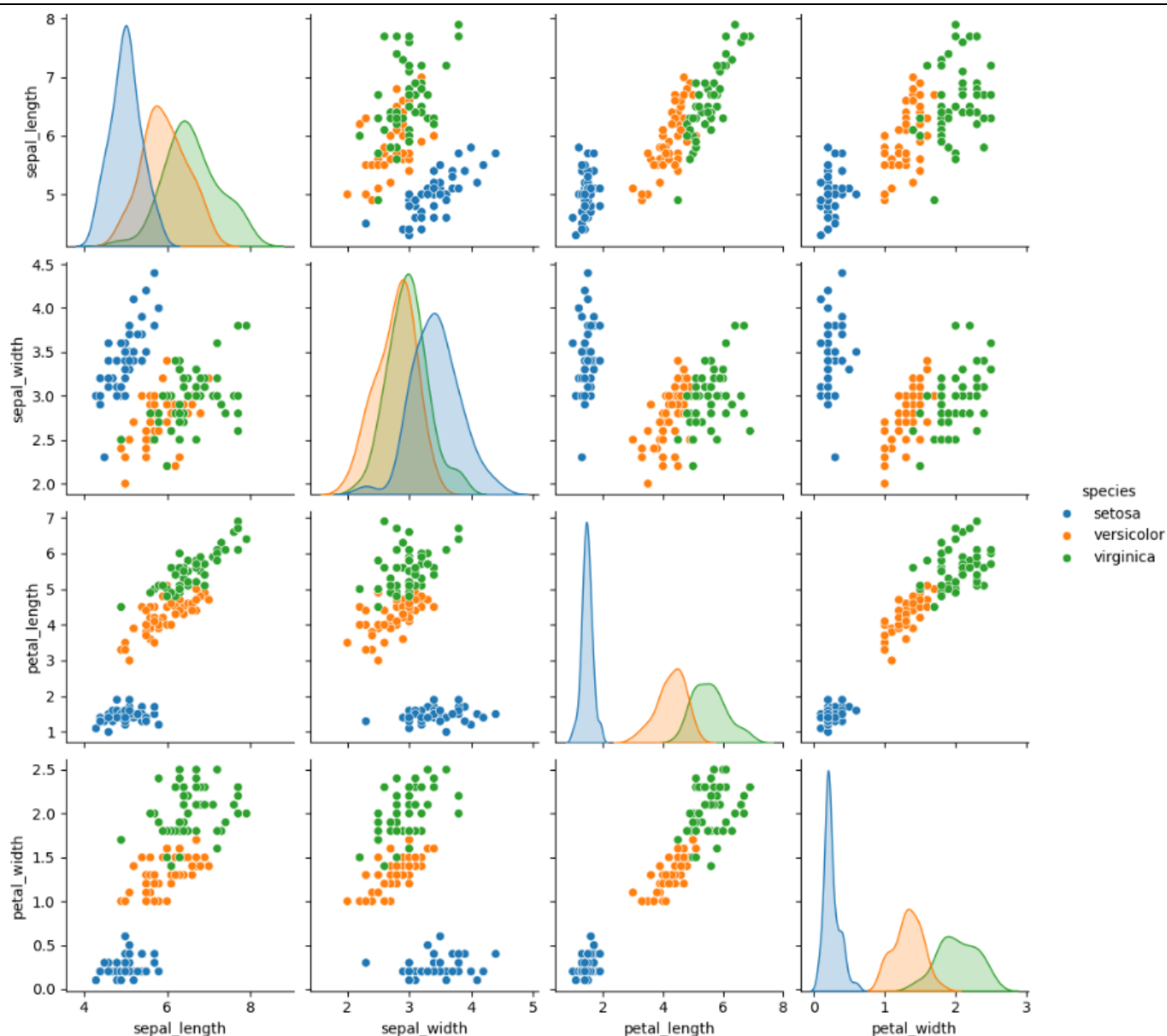
5.2.5 Pairplots

Los pairplots comparan gráficamente las relaciones entre cada par de características numéricas. Son útiles para identificar agrupaciones, patrones y relaciones lineales o no lineales.

Ejemplo en Python:

```
import seaborn as sns
import pandas as pd

df = sns.load_dataset('iris')
sns.pairplot(df, hue='species')
```



5.2.6 Significancia Estadística en el Análisis Bivariado

Observar visualmente la relación entre dos variables es solo el comienzo. Es importante validar estadísticamente estas relaciones para asegurarse de que no sean simplemente patrones aleatorios. Este paso es crucial para obtener conclusiones confiables.

Hay diferentes pruebas estadísticas que se pueden utilizar para este fin, dependiendo de la naturaleza de las variables involucradas.

Por ejemplo, la prueba de **Pearson** se puede utilizar para medir la fuerza y dirección de la correlación entre dos variables numéricas. Del mismo modo, la prueba de **Spearman** se puede utilizar para medir la correlación entre dos variables ordinales o cuando no se puede asumir una comprensión más profunda de la relación entre las variables. Estas pruebas nos permiten identificar patrones en los datos con un cierto nivel de significancia.

Ejemplo en Python usando `scipy.stats` para verificar la correlación de Pearson y su significancia:

```
from scipy import stats

# Ejemplo simple
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

# Prueba de correlación de Pearson
corr, p_value = stats.pearsonr(x, y)

print("Correlation:", corr)
print("P-value:", p_value)
```

```
Correlation: 1.0
P-value: 0.0
```

El **p-valor** indicará si la correlación es estadísticamente significativa. Generalmente, se considera que un valor p menor que 0.05 indica significancia estadística.

5.2.7 Manejo de Variables Categóricas en el Análisis Bivariado

Cuando una variable es numérica y otra es categórica, los **diagramas de caja (box plots)** son una excelente opción para ofrecer información visual. Por ejemplo, al estudiar la relación entre el tipo de variable categórica y crear un diagrama de caja para ver cómo varía la distribución de la variable numérica en diferentes categorías.

Además, podemos agregar medidas estadísticas como la **mediana**, los **cuartiles** o los **valores atípicos** para obtener una visión más completa de los datos. También podemos realizar pruebas estadísticas como el **análisis de varianza (ANOVA)** o la **prueba de Kruskal-Wallis** para ver si existen diferencias significativas entre los grupos.

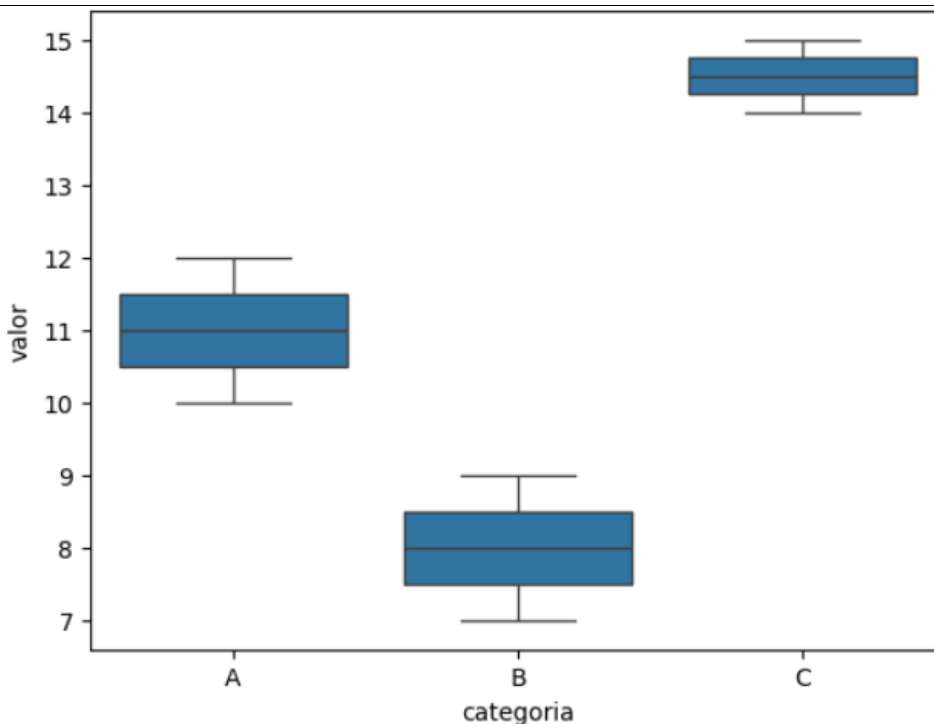
Ejemplo usando Seaborn para generar un diagrama de caja:



```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Creamos un DataFrame de ejemplo
datos = pd.DataFrame({
    "categoria": ["A", "A", "B", "B", "C", "C"],
    "valor": [10, 12, 7, 9, 14, 15]
})

# Boxplot
sns.boxplot(x="categoria", y="valor", data=datos)
plt.show()
```



5.2.8 Aplicaciones del Análisis Bivariado en el Mundo Real

El análisis bivariado no es solo una habilidad para datos. La capacidad de analizar la relación entre dos variables puede tener aplicaciones en el mundo real en una variedad de campos. Por ejemplo:

- Cómo el ingreso afecta el nivel educativo
- Cómo la edad influye en el comportamiento de compra
- Cómo el clima afecta la producción agrícola

Estas aplicaciones pueden proporcionar información valiosa para la toma de decisiones, el desarrollo de estrategias y la mejora de procesos. Al comprender cómo dos variables están relacionadas, podemos identificar patrones, hacer predicciones y desarrollar ideas.



Además, el análisis bivariado puede ser una herramienta poderosa para los científicos de datos. En el análisis exploratorio de datos, puede ayudar a identificar relaciones importantes que deben ser exploradas más a fondo. También puede ser útil para preparar datos para el modelado, al seleccionar características relevantes y eliminar aquellas que no aportan valor.

En resumen, el análisis bivariado es una herramienta poderosa para entender mejor nuestros datos y tomar decisiones informadas. Ya sea en el mundo académico, empresarial o científico, el análisis bivariado puede ser aplicado para adquirir una comprensión más profunda de los datos y mejorar los resultados.

5.3 ANÁLISIS MULTIVARIADO

A medida que nos adentramos más en los reinos de los datos, llegamos a un punto en el que el análisis bivariado ya no es suficiente. En el mundo real, los datos son complejos y están interrelacionados. Para entender esta nueva y emocionante dimensión de los datos, necesitamos el **análisis multivariado**.

El análisis multivariado nos permite estudiar múltiples variables simultáneamente, lo que nos ayuda a comprender mejor la complejidad del análisis de datos. En esta sección, vamos a explorar el análisis multivariado, incluyendo el concepto de estudiar relaciones por pares, y cómo podemos utilizar herramientas como **PCA** y **análisis de clústeres**.

5.3.1 ¿Qué es el Análisis Multivariado?

El análisis multivariado es una rama de la estadística que implica el estudio simultáneo de múltiples variables. A diferencia del análisis bivariado, que examina la relación entre dos variables, el análisis multivariado considera múltiples variables al mismo tiempo para entender cómo interactúan entre sí.

Este tipo de análisis es especialmente útil cuando se trabaja con conjuntos de datos grandes y complejos, donde las relaciones entre variables pueden ser profundas y difíciles de identificar. El análisis multivariado nos permite descubrir patrones ocultos, identificar relaciones importantes y segmentar los datos de manera efectiva.

Beneficios principales:

- **Reducción de dimensionalidad**
- **Identificación de patrones ocultos**
- **Segmentación de datos**
- **Mejora de modelos predictivos**

Se utiliza en estudios de mercado, análisis financiero, investigaciones científicas y desarrollo de productos.



5.3.2 Tipos de Análisis Multivariado

Técnicas más comunes:

- **Análisis de Componentes Principales (PCA):** técnica de reducción de dimensionalidad que representa relaciones entre variables en un espacio de menor dimensión.
- **Análisis de Clústeres:** agrupa datos en segmentos basados en similitudes.
- **Regresión Múltiple:** predice una variable dependiente a partir de múltiples independientes.

5.3.3 Ejemplo: Análisis de Componentes Principales (PCA)

Ejemplo con scikit-learn:

```
from sklearn.decomposition import PCA
import numpy as np

X = np.array([[1, 2], [3, 4], [5, 6], [7, 8]])

pca = PCA(n_components=1)
X_pca = pca.fit_transform(X)

print("Varianza explicada:", pca.explained_variance_)
print("Componentes:", pca.components_)

# explained_variance_ indica cuánta información (varianza)
# puede atribuir cada componente principal.
```

Varianza explicada: [13.33333333]

Componentes: [[0.70710678 0.70710678]]

Desde el punto de vista del "detective de datos", estos resultados sugieren que tienes dos variables que están **fuertemente relacionadas** y que pueden resumirse en un único valor (el componente principal) que retiene una varianza de 13.33 unidades. Esto permite la **reducción de dimensionalidad**, simplificando el análisis posterior sin perder la estructura esencial de los datos

5.3.4 Ejemplo: Análisis de Clústeres

Así se realiza un análisis de clústeres k-means con scikit-learn:

```
import numpy as np
from sklearn.cluster import KMeans

# Datos realistas: [ingresos mensuales, gasto en la tienda]
X = np.array([
    [1200, 150],
    [1300, 200],
```



```
[1250, 180],
[3000, 450],
[3200, 500],
[2800, 400],
[6000, 900],
[5800, 850],
[6200, 1000],
[1500, 220],
[2900, 380],
[6100, 950]
])

# Creamos el modelo con 3 clústeres
kmeans = KMeans(n_clusters=3)
kmeans.fit(X)

# Resultados
centroides = kmeans.cluster_centers_
etiquetas = kmeans.labels_

print("=== RESULTADOS DEL MODELO K-MEANS ===\n")

print("Centroides encontrados:")
print(centroides)
print("\nInterpretación:")
print("- Cada fila representa el centro de un clúster.")
print("- El primer valor es el ingreso medio del grupo.")
print("- El segundo valor es el gasto medio del grupo.\n")

print("Etiquetas asignadas a cada punto:")
print(etiquetas)
print("\nInterpretación:")
print("- Cada número indica a qué clúster pertenece cada cliente.")
print("- Clientes con la misma etiqueta comparten un patrón similar.\n")

print("Conclusión general:")
print("- El algoritmo ha identificado tres tipos de clientes:")
print(" * Un grupo de ingresos y gasto bajos.")
print(" * Un grupo de ingresos y gasto medios.")
print(" * Un grupo de ingresos y gasto altos.")
print("- Los centroides resumen el comportamiento típico de cada grupo.")
```

RESULTADO DE LA EJECIÓN:

=== RESULTADOS DEL MODELO K-MEANS ===

Centroides encontrados:



[[1312.5 187.5]
[6025. 925.]
[2975. 432.5]]

Interpretación:

- Cada fila representa el centro de un clúster.
- El primer valor es el ingreso medio del grupo.
- El segundo valor es el gasto medio del grupo.

Etiquetas asignadas a cada punto:

[0 0 0 2 2 2 1 1 1 0 2 1]

Interpretación:

- Cada número indica a qué clúster pertenece cada cliente.
- Clientes con la misma etiqueta comparten un patrón similar.

Conclusión general:

- El algoritmo ha identificado tres tipos de clientes:
 - * Un grupo de ingresos y gasto bajos.
 - * Un grupo de ingresos y gasto medios.
 - * Un grupo de ingresos y gasto altos.
- Los centroides resumen el comportamiento típico de cada grupo.

La variable **centroids** contiene las coordenadas de los centros de los clústeres, y **labels** contiene las etiquetas de categoría para cada punto de datos.

5.3.5 Aplicaciones del Análisis Multivariado en el Mundo Real

El análisis multivariado es una **herramienta poderosa** que ha encontrado su lugar en muchas industrias. En el cuidado de la salud, se puede utilizar para examinar las **complejas relaciones entre varios marcadores biológicos**, lo que permite a los investigadores descubrir nuevos conocimientos sobre el cuerpo humano y las enfermedades. En finanzas, el análisis multivariado puede ayudarnos a comprender mejor las **intrincadas relaciones entre diferentes variables financieras**, como el riesgo, el ROI y las tendencias del mercado, lo que nos permite tomar decisiones más informadas al gestionar nuestras carteras.

Sin embargo, es importante tener en cuenta que si bien el análisis multivariado puede proporcionar conocimientos sólidos, no es una solución milagrosa. Como con cualquier método estadístico, es importante ser **cauteloso y estar atento a la calidad de sus datos** y a las suposiciones detrás del análisis.

Uno siempre debe ser consciente de las posibles limitaciones de su análisis y tomar medidas para mitigar cualquier sesgo potencial o fuentes de error. Por lo tanto, es importante abordar el análisis



multivariado con un **ojo crítico** y evaluar y refinar constantemente sus métodos a medida que se disponga de nuevos datos y conocimientos.

5.3.6 Mapas de Calor para Matrices de Correlación

El **mapa de calor** es una herramienta gráfica increíble que puede ayudarnos a comprender la relación entre múltiples variables. Podemos usarlo para mostrar los datos en un formato **codificado por colores para identificar patrones y tendencias**. Python proporciona muchas bibliotecas que pueden ayudarnos a crear mapas de calor sorprendentemente hermosos e informativos, como **Seaborn**.

Con Seaborn, podemos personalizar el esquema de colores, las anotaciones y otros parámetros para obtener la salida deseada. Además, también podemos usar mapas de calor interactivos que nos permiten hacer zoom dentro y fuera, pasar el cursor sobre y hacer clic en las celdas para obtener más información. Al explorar los mapas de calor, podemos obtener **información valiosa sobre los datos** y tomar decisiones informadas basadas en los hallazgos.

Ejemplo:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

np.random.seed(42)

# Variables con sentido
n = 120

# Relación positiva fuerte: más horas de estudio → mejor nota
horas_estudio = np.random.normal(4, 1.2, n).clip(0, 10)
nota_examen = horas_estudio * 1.5 + np.random.normal(0, 1, n)
nota_examen = nota_examen.clip(0, 10)

# Relación negativa fuerte: más videojuegos → peor nota
horas_videojuegos = 10 - horas_estudio + np.random.normal(0, 1, n)
horas_videojuegos = horas_videojuegos.clip(0, 12)

# Relación ≈ 0: altura no tiene nada que ver con estudiar o notas
altura_cm = np.random.normal(165, 10, n)

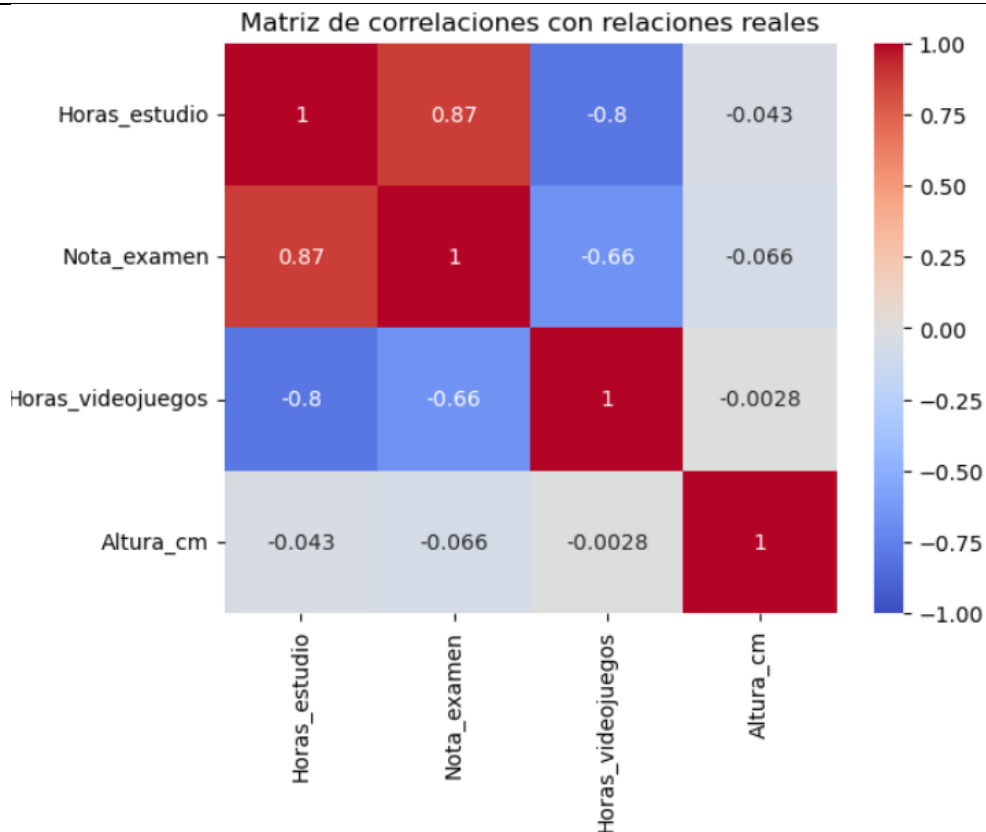
# Crear DataFrame
df = pd.DataFrame({
    'Horas_estudio': horas_estudio,
    'Nota_examen': nota_examen,
    'Horas_videojuegos': horas_videojuegos,
```




```
'Altura_cm': altura_cm
})

# Matriz de correlación
corr = df.corr()

# Heatmap
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Matriz de correlaciones con relaciones reales")
plt.show()
```



En este mapa de calor, cada cuadrado muestra la correlación entre dos características. Un valor cercano a 1 implica una **fuerte correlación positiva**: a medida que una característica aumenta, la otra característica tiende a aumentar también. Un valor cercano a -1 implica una **fuerte correlación negativa**: a medida que una característica aumenta, la otra característica tiende a disminuir.

5.3.7 Ejemplo utilizando Análisis de Regresión Múltiple

El análisis multivariado es una **técnica estadística poderosa** que permite a los investigadores explorar las complejas relaciones entre múltiples variables. Un ejemplo de ello es la **regresión múltiple**, que es particularmente útil cuando se intenta comprender cómo dos o más variables independientes se relacionan con una única variable dependiente.



Al analizar el impacto de cada variable independiente en el resultado, podemos obtener una comprensión más profunda de los factores subyacentes que influyen en la variable dependiente. A través de este método, los investigadores pueden generar conocimientos más matizados sobre fenómenos del mundo real, como el comportamiento del consumidor, las tendencias económicas y las dinámicas sociales, entre otros.

En última instancia, la capacidad de realizar análisis multivariados es una herramienta valiosa en el arsenal de cualquier investigador, ya que nos permite obtener **conclusiones más precisas a partir de conjuntos de datos complejos** y desarrollar soluciones más efectivas para problemas del mundo real.

Ejemplo:

```
import statsmodels.api as sm

# Generating some example data
np.random.seed(42)
X = np.random.randn(100, 3) # 3 features
y = 2 * X[:, 0] + 1.5 * X[:, 1] + 0.7 * X[:, 2] + np.random.randn(100) # dependent variable

# Fitting multiple linear regression
X_with_const = sm.add_constant(X)
model = sm.OLS(y, X_with_const).fit()

# Summary of regression, including both univariate and multivariate statistics
print(model.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:          0.848
Model:                  OLS    Adj. R-squared:      0.843
Method:                 Least Squares    F-statistic:      178.4
Date:                   Sun, 15 Feb 2026    Prob (F-statistic): 4.02e-39
Time:                   18:22:49    Log-Likelihood:   -127.96
No. Observations:       100    AIC:              263.9
Df Residuals:           96    BIC:              274.3
Df Model:                3
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.1129	0.091	1.241	0.218	-0.068	0.293
x1	1.9223	0.109	17.602	0.000	1.706	2.139
x2	1.4500	0.092	15.755	0.000	1.267	1.633
x3	0.5924	0.081	7.271	0.000	0.431	0.754

```

=====
Omnibus:                0.771    Durbin-Watson:          1.891
Prob(Omnibus):           0.680    Jarque-Bera (JB):        0.847
Skew:                    0.198    Prob(JB):                0.655
Kurtosis:                2.785    Cond. No.:               1.47
=====

```

Aquí tienes un **resumen muy breve y claro** del resultado de la regresión múltiple:



El modelo explica **muy bien** la variable y ($R^2 \approx 0.85$). Las tres variables **x1, x2 y x3 son significativas** y contribuyen realmente a predecir y:

- **x1** es la que más influye
- **x2** también tiene un efecto fuerte
- **x3** tiene un efecto más pequeño, pero real

Los diagnósticos indican que:

- Los residuos son **normales**
- No hay **autocorrelación**
- No existe **multicolinealidad**
- El modelo está **bien ajustado y es estadísticamente sólido**

En conjunto: **Es un modelo fiable, estable y con muy buena capacidad explicativa.**

5.3.8 Puntos de Precaución

1. **Sobreajuste:** Incluir demasiadas variables puede hacer que tu modelo sea demasiado complejo, lo que podría hacer que funcione bien en los datos de entrenamiento pero mal en datos nuevos y no vistos. Una forma de abordar el sobreajuste es mediante el uso de **técnicas de regularización como la regularización L1 o L2**, que pueden ayudar a reducir el impacto de las variables menos importantes y prevenir el sobreajuste. Otro enfoque es utilizar la **validación cruzada** para evaluar el rendimiento de tu modelo en nuevos datos.
2. **Multicolinealidad:** Esto ocurre cuando las variables independientes están altamente correlacionadas entre sí, lo que dificulta aislar el efecto de cada variable. Puedes utilizar técnicas como el **Factor de Inflación de la Varianza (VIF)** para detectar la multicolinealidad. Una vez identificada la multicolinealidad, puedes considerar eliminar una o más de las variables altamente correlacionadas de tu modelo. Alternativamente, puedes utilizar técnicas como el **análisis de componentes principales (PCA)** para reducir la dimensionalidad de tu conjunto de datos y abordar la multicolinealidad.

5.3.9 Otras Técnicas de Reducción de Dimensionalidad

Además de PCA, existen otras técnicas de reducción de dimensionalidad que son útiles en el análisis multivariado:

1. **t-SNE** (t-Distributed Stochastic Neighbor Embedding): Este algoritmo es una herramienta poderosa para **visualizar datos de alta dimensionalidad**. Funciona mapeando los datos a un espacio de menor dimensionalidad donde se pueden visualizar y analizar fácilmente. t-SNE es



particularmente útil cuando se trata de conjuntos de datos complejos que no pueden entenderse fácilmente utilizando técnicas de visualización tradicionales.

2. **UMAP** (Uniform Manifold Approximation and Projection): Similar a t-SNE, UMAP es otra forma eficiente de reducir la dimensionalidad de datos de alta dimensionalidad. Funciona proyectando los datos en un espacio de menor dimensionalidad mientras preserva la estructura topológica de los datos originales. Esto lo convierte en una opción popular para la visualización de datos, especialmente en casos donde los datos son demasiado complejos para ser visualizados fácilmente utilizando técnicas tradicionales.

Así es como podrías aplicar t-SNE usando la biblioteca scikit-learn:

```
from sklearn.manifold import TSNE
```

```
X_embedded = TSNE(n_components=2).fit_transform(X)
```

NOTA: Instalar UMAP con pip

```
pip install umap-learn
```

Y para UMAP, puedes usar la biblioteca umap-learn de la siguiente manera:

```
import umap
```

```
reducer = umap.UMAP()
```

```
X_embedded = reducer.fit_transform(X)
```

EJEMPLOS PRÁCTICOS CAPÍTULO 5

Ejercicio 1: Análisis Univariado con Histogramas

Tarea: Dado un conjunto de datos de puntuaciones de exámenes para estudiantes, traza un **histograma para entender la distribución de las puntuaciones**.

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
import numpy as np
```

```
# Generamos una variable con sentido: horas de estudio de 100 estudiantes
```

```
np.random.seed(42)
```

```
horas_estudio = np.random.normal(loc=4, scale=1.5, size=100).clip(0, 10)
```

```
# Creamos el DataFrame
```

```
df = pd.DataFrame({
```

```
    'Horas_estudio': horas_estudio
```

```
})
```



Histograma (análisis univariante)

```
plt.figure(figsize=(8,5))
```

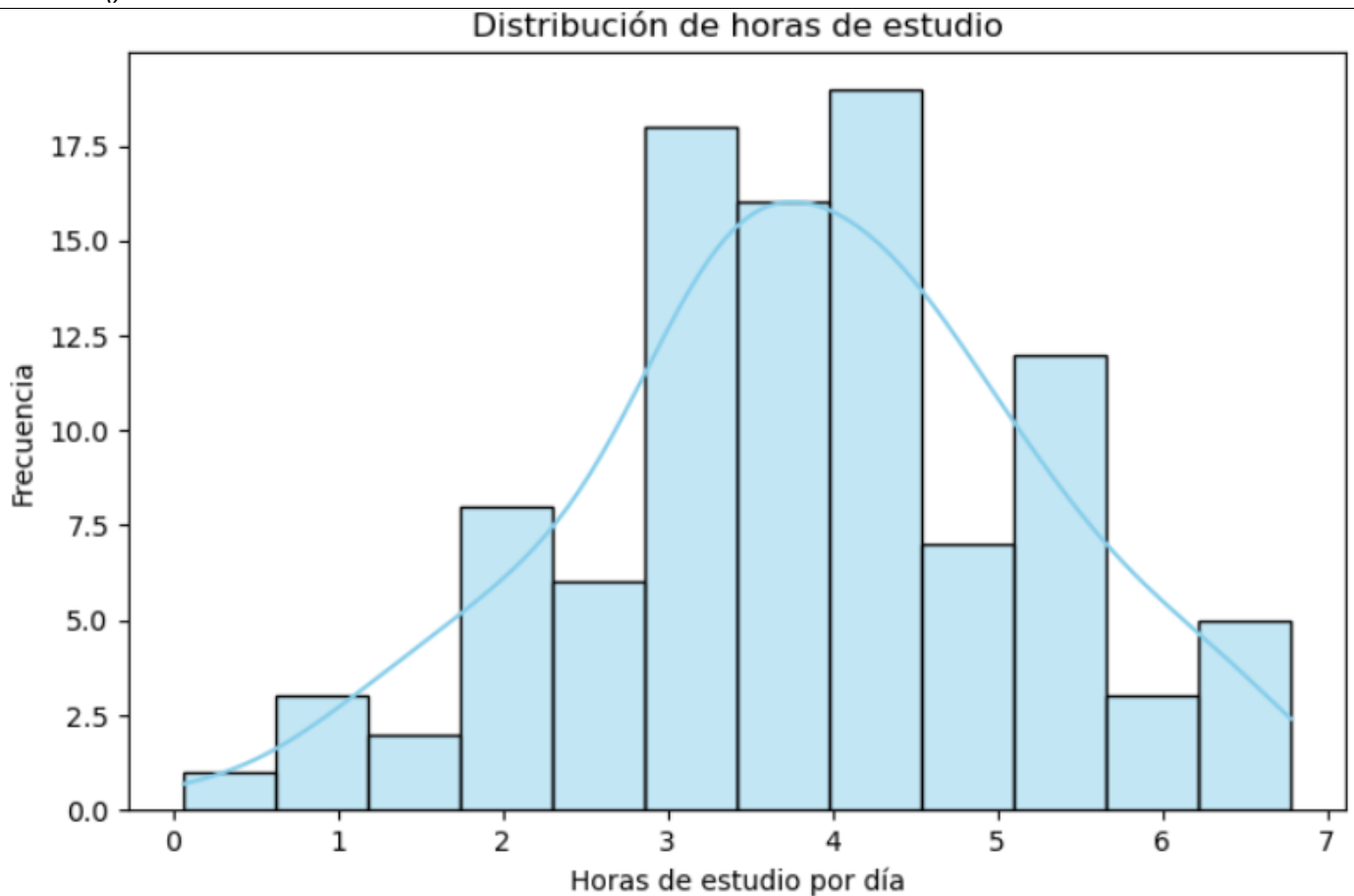
```
sns.histplot(df['Horas_estudio'], bins=12, kde=True, color='skyblue')
```

```
plt.title("Distribución de horas de estudio")
```

```
plt.xlabel("Horas de estudio por día")
```

```
plt.ylabel("Frecuencia")
```

```
plt.show()
```



El histograma muestra la **distribución de las horas de estudio por día** en un grupo de estudiantes. A partir de la forma del gráfico, podemos extraer varias conclusiones:

1. Distribución aproximadamente normal

La forma es parecida a una campana:

- La mayoría de las estudiantes estudia entre **3 y 5 horas al día**.
- Hay menos estudiantes que estudian muy poco o mucho.

Esto indica que el comportamiento es bastante homogéneo.

2. Valor central claro

El pico del histograma está alrededor de **4 horas**, lo que sugiere que:

La cantidad típica de estudio diario se sitúa en torno a 4 horas.



3. Variabilidad moderada

Los valores se extienden aproximadamente entre **0 y 8 horas**, lo que muestra cierta variación, pero sin extremos exagerados.

4. Ausencia de valores atípicos fuertes

No se observan barras aisladas en los extremos, por lo que no parece haber estudiantes con hábitos extremadamente distintos al resto.

5. La curva KDE confirma la forma

La línea suave superpuesta refuerza la idea de una distribución:

- **Simétrica**, sin sesgos marcados.
- **Concentrada** en torno a la media.

Conclusión breve

La variable *horas de estudio* presenta una distribución normal, centrada en torno a 4 horas, con variabilidad moderada y sin valores extremos. Es un patrón típico en variables de comportamiento humano.

Ejercicio 2: Análisis Bivariado con Gráfico de Dispersión

Tarea: Crea un gráfico de dispersión para visualizar la **relación entre las alturas y los pesos de un grupo de individuos**.

```
import numpy as np
import matplotlib.pyplot as plt

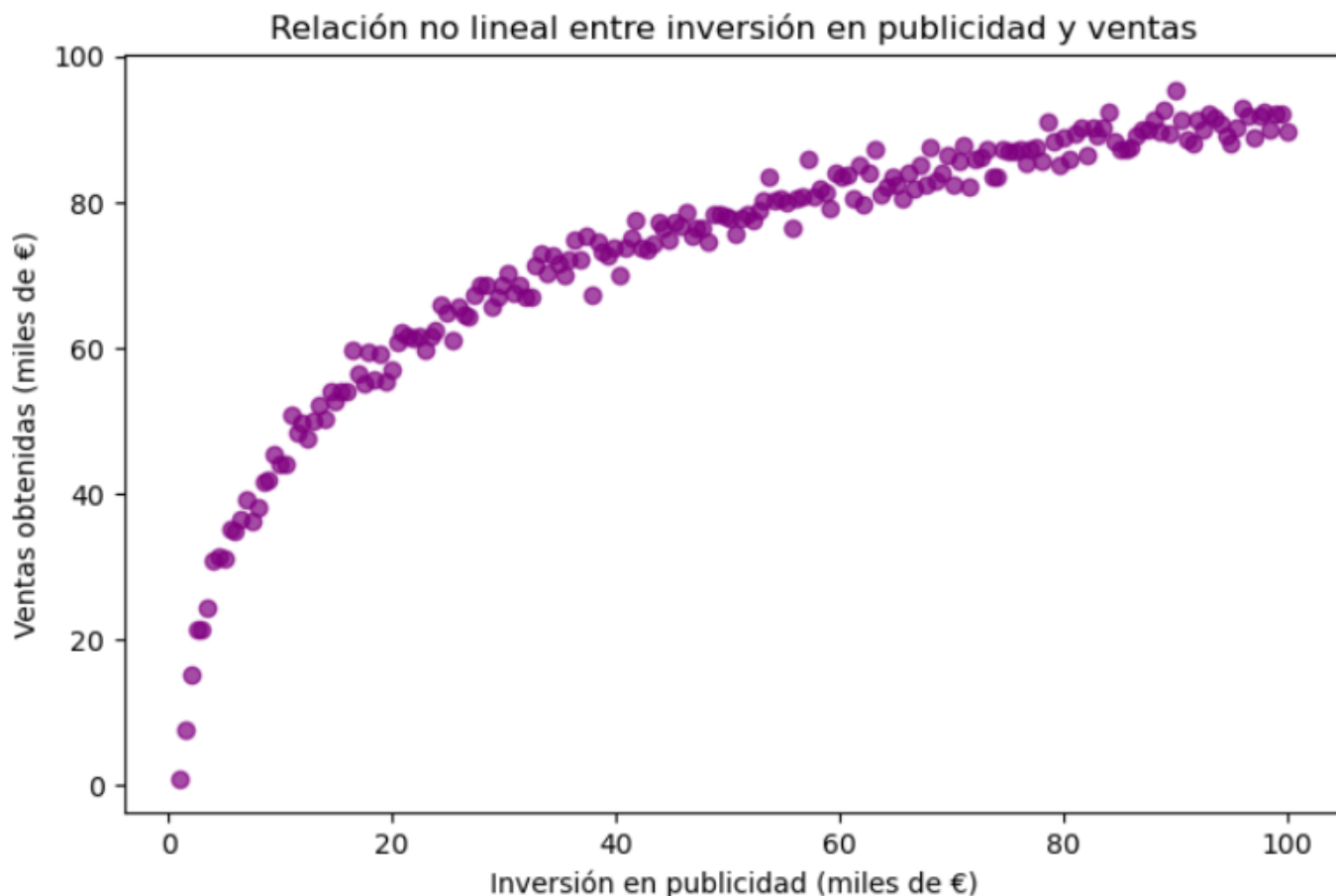
np.random.seed(42)

# Variable independiente: inversión en publicidad (en miles de euros)
x = np.linspace(1, 100, 200)

# Relación logarítmica: ventas que crecen rápido al principio y luego se estabilizan
y = 20 * np.log(x) + np.random.normal(0, 2, 200)

# Gráfico de dispersión
plt.figure(figsize=(8,5))
plt.scatter(x, y, alpha=0.7, color='purple')
plt.xlabel("Inversión en publicidad (miles de €)")
plt.ylabel("Ventas obtenidas (miles de €)")
plt.title("Relación no lineal entre inversión en publicidad y ventas")
plt.show()
```





Ejercicio 3: Análisis Multivariado usando un Mapa de Calor

BIBLIOGRAFÍA

1. Fundamentos del Análisis de Datos con Python: <https://www.quantum.tech/es/learning-path/ai-engineering>

2. Curso introductorio en Google Colab (gratuito y práctico)

Un curso abierto creado por estudiantes del IEEE, ideal para empezar a manipular datos, automatizar tareas y visualizar información.

https://colab.research.google.com/github/IEEESBITBA/Curso-python/blob/master/Curso_Analisis_de_Datos_Clases/Clase_1_Analisis_de_Datos.ipynb#scrollTo=-0006-ePnUur

3. freeCodeCamp – Análisis de Datos con Python

Uno de los recursos más completos y gratuitos. Enseña NumPy, Pandas, Matplotlib y Seaborn con ejercicios reales.



<https://www.freecodecamp.org/espanol/learn/data-analysis-with-python/#data-analysis-with-python-course>

4. Curso de Análisis de Datos Gratis (Python, Excel y R)

Curso certificado que cubre fundamentos del análisis de datos, limpieza, preparación y técnicas estadísticas básicas.

<https://videocursos.co/curso-de-analisis-de-datos-con-python/>

5. Repositorio “Awesome Data Analysis” (GitHub)

Colección de más de 500 recursos gratuitos sobre análisis de datos, Python, estadística y visualización.

